Few-shot Classification for Real-time Additive Manufacturing Fault Detection on Edge Devices

Oliver-James Bravery | C2020447 | Supervisor: Tong Xin

Word Count: 16483 (Microsoft Word)

Abstract

In this project, I successfully demonstrate the effectiveness of training a few-shot classification model over small datasets to produce well-generalisable fault detection models for the additive manufacturing process. Investigations into different network architectures revealed that using a prototypical network with a lightweight feature extractor (ShuffleNetv2) achieves SOTA (state-of-the-art) performance across a variety of datasets for fully open-source edge-deployable fault detection systems, illustrating the model's ability to generalise well across the domain. Evaluations into different training configurations revealed fine-tuning the feature extractor negatively affected performance of the overall model. Assessing the model's throughput, precision and recall (F1 score) on an edge device (Raspberry Pi 4B) reveals real-time capability, with an image throughput of 15.1 images per second alongside an average F1 score across datasets of 93.7%, markedly outperforming existing models.

Table of Contents

1 Introduction

1.1 Context

Additive manufacturing (AM), commonly referred to as 3D-printing has become a rapidly advancing manufacturing technique in the past decade, with recent advances lowering the threshold, enabling individuals, businesses, and industries to adopt the technology with a wide range of affordable and high-end machines. One of the most common AM systems is fused deposition modelling (FDM) where filament is extruded layer by layer to form the desired object.

AM is becoming more popular amongst individuals and hobbyists as printers becoming cheaper and less complicated to operate as emerging companies such as Bambu Labs release pre-built FDM (fusion deposition modelling) printers such as the A1-mini for less than £200, built with auto calibration and other handy features making machines more accessible (Bambu Labs, no date). A press release from the market intelligence company Context found in 2023, a "*7% year-on-year increase in global 3D printer revenues for the quarter was driven entirely by the surge in ENTRY-LEVEL shipments, with revenues 58% higher than in Q2 2023*" (ContextWorld, 2024), demonstrating the popularity of 3D printing with hobbyists and individuals.

Whilst the 3D printer industry evolves and the technologies improve, they are still prone to errors whilst printing. There are many different types of well documented print defects which can cause prints to fail, such as under extrusion issues which occur due to slicing issues or clogged and leaky extrusion nozzles, causing material to be missing in print layers, leading to weak or incomplete prints. Another common issue is named the spaghetti monster, which "*looks exactly like it sounds, a big mess of "spaghetti" on and around your print*" (Prusa Research, no date), typically caused by prints detaching from the print bed and moving during the printing process or a print collapsing, causing the extruder to print filament into thin air.

It is crucial to catch these errors early on to prevent wasting excess filament and energy on completing a faulty print. Furthermore, certain issues such as under extrusion errors where the nozzle becomes jammed, if not caught early, may cause the nozzle to become fully defective, thus breaking the machine. With some prints taking multiple days to complete, its in-feasible for a person to constantly observe the process for errors which is where automatic fault detection systems are employed. These systems capture what's being printed and utilises neural networks to classify the print as defective or acceptable, notifying the user or automatically pausing the print if errors are detected to prevent breakages and waste.

1.2 The Problem

Whilst fault detection systems do exist, they are typically large, closed source models, hosted privately behind costly APIs. As businesses, hobbyists and industry leaders all utilise additive manufacturing for their own private needs, and these machines may be in environments where privacy is paramount, it's imperative that the deployment of these oversight systems are secure, private and reliable. For businesses printing products for customers for example, they may not be able to use cloud-based fault detection system which analyse footage due to the sensitivity of what's being printed.

Furthermore, as a substantial number of these detection systems are only accessible via costly APIs such as Obico's '*spaghetti detective*' costing $6 USD a month for 50 hours of detection (Obico, no date a), consumers such as hobbyists with a limited budget are left without a way to detect print issues. This emphasises the need for on-device, self-deployable edge systems, which gives the end user full control over their data, at little to no cost.

Additive manufacturing fault datasets are scarce (as reviewed in section 3.5) meaning the architecture used in my neural network must be capable of training effectively on small datasets. "*Few-shot learning is a machine learning framework in which an AI model learns to make accurate predictions by training on a very small number of labeled examples*" (IBM, 2024), learning to classify based on a few examples per class. They're often training through a type of metric-learning instead of traditional transfer-learning, where the model trains

across many tasks to efficiently adapt to new ones (learning to learn) instead of reusing knowledge from one domain to another (transfer learning used in fine-tuning for example). As research into using few-shot networks for real-time edge deployable systems is limited, I will be investigating whether few-shot classification can be used to produce a real-time edge deployable model.

2 Aim and Objectives

2.1 High-Level Aim

Here I aim to investigate and produce a real-time, edge-deployable additive manufacturing fault detection system with a few-shot classification backbone which can be inferred on common, low-cost devices, enabling private, secure and affordable print monitoring. The investigation also evaluates whether few-shot classification is a suitable approach for real-time, edge deployment by measuring performance, generalisation, and computational efficiency on a low-resource device.

2.2 Objectives

Objective 1: Research existing additive manufacturing fault detection systems, identifying their strengths and limitations

I aim to build an understanding of what systems exist for AM fault detection, the hardware they require to function and evaluating their performance. Recognizing the strengths and weakness of current studies and existing fault systems is imperative so that gaps in these solutions can be explored and effective strategies found can be built upon. I will also need to assess what datasets these techniques used during training, analysing their licences and quality. Reviewing the diversity of each dataset is imperative, ensuring they contain a vast amount of differing camera angles, lighting scenarios and print material colours, assuring any model trained on it would be able to generalise well across the domain.

Objective 2: Investigate few-shot neural network architectures and techniques to allow real-time inference on edge devices

Examine existing neural networks, assessing architectures for training with small datasets. Evaluate how well they perform on edge devices in relation to their inference speeds for real-time applications. This objective could be met by finding and analysing existing studies for few-shot classification alongside reviewing their related papers. I aim to identify the most appropriate few-shot neural network to be used in my fault detection model, assessing its applicability in relation to transfer-learning based networks for real-time edge deployment.

Objective 3: Fine-tune a pre-trained image encoder for domain-specific feature extraction in 3D-printing image data

For neural networks that process images (i.e. frames from a camera) typically, an encoder such as a CNN (convolutional neural network) is employed to embed the image as processable data for the chosen architecture. Due to time constraints, training a CNN is out of this projects scope. This objective seeks to investigate and adapt a pre-trained CNN (e.g., UNET) originally trained on a large-scale dataset (e.g., ImageNet), to enhance its specialisation in the 3D-printing domain. Here I aim to produce an encoder for the system, fine-tuned for additive manufacturing fault detection.

Objective 4: Train and evaluate an additive manufacturing fault detection model utilising the previously established research and techniques

Utilising the prior research into few-shot classification architectures, leveraging the trained image encoder from previous objectives, implement and train a network for AM defect detection. This objective aims to produce a model which can classify an image of a 3D print as either defective or non-defective. The model should be evaluated based on its practical inference speed on a well know edge device (i.e. Raspberry Pi), alongside measuring its accuracy on its validation and test dataset's (i.e. F1 score and precision).

3 Background Review

3.1 Additive Manufacturing Common Issues

It is important to understand what issues 3D printers can encounter and the effect they have on the print. We only want my network to detect issues which may lead to failed prints or machine breakages, not temporary issues which may result in slight resolvable artifacts, such as stringing.

3.1.1 Stringing and Oozing

Stringing and oozing are common artifacts when printing, presented by thin strings of filament being left behind and stretching across the print. As described by the 3D printing firm MatterHackers, they are typically caused by "*slow extruder movement[s] between sections of a part or more than one part*" of the print (MatterHackers, 2025), printing across a gap or a filament retraction issue, where the extruder fails to retract enough filament, leaving excess on the print. Whilst this issue does cause the print to look malformed, it is unlikely that stringing will cause breakages to the printer as it is just extra extruded filament which "*can be removed rather quickly with a lighter*" (SimplyPrint, 2020). As this issue is not detrimental, including this type of error when training my model would not be beneficial as my model should only identify defects which could cause the print to fail.

Fig. 1 - Stringing defect in 3D print (Simplify3D, 2019a)

3.1.2 Improper Bed Adhesion (Spaghetti Monster)

When printing objects, the first layer of the print is temporarily bonded to the printers build platform (bed) by either, heating the bed to a consistent temperature or by using an adhesion promoter such as glue. During the printing process however, there is a possibility that the bed may lose temperature and the extruders nozzle may catch the print, nudging it or, if the build platform isn't level, "*one side of your bed may be too close to the nozzle, while the other side is too far away*", leading to improper bed adhesion (Simplify3D, 2019b). If the print moves during extrusion, the filament may deposit in wrong positions, causing *spaghetti monsters*, "*a big mess of "spaghetti" on and around your print"* (Prusa Research, no date). This issue is often detrimental to the final product, where a spaghetti monster makes the final print indistinguishable. Due to the impact of this issue, *spaghetti monsters* should be identified by my defect detection network so that filament isn't wasted printing unnecessary filament spaghetti.

Fig. 2 – Improper bed adhesion defect in 3D print (Prusa Research, no date)

3.1.3 Warping

Warping is an artifact encountered where an "*edge near the bottom of the print or surface adjacent to the print bed is not level or flat*" (MatterHackers, 2025), leading to filament in later layers extruding in incorrect places. If the bottom layer of a print warps, it may improper bed adhesion and subsequently, spaghetti monsters. Warping is presented by small surface edges not being flat, making it difficult for a camera to pick up on those issues. It may be more suitable to train my fault detection system to detect the issues caused by warping rather than warping itself, since these are more visually distinct and detectable by a camera-based system.

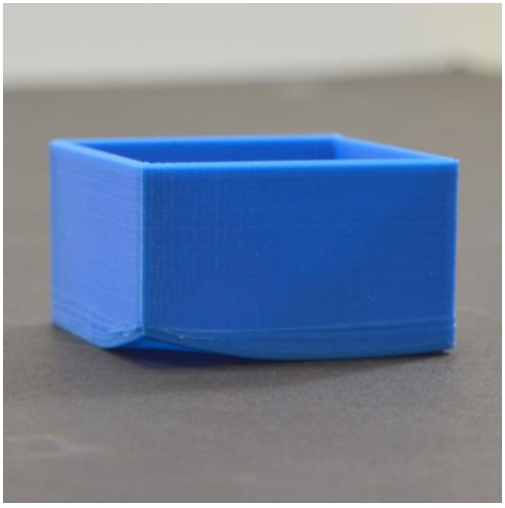

Fig. 3 – Warping defect in 3D print (Simplify3D, 2019c)

3.1.4 Layer Separation and Splitting

3D printing extrudes filament layer by layer, using heat to bind layers. If the temperature is too low, or the nozzle is too far from the previous layer, layers may not adequately bond due to filament cooling prior to bonding. Whilst this error can cause similar issues to improper bed adhesion, such as spaghetti monsters, prints are still capable of completing, making them more visual than detrimental defects. Some prints may have similar looking designs to the separated layers issue, which could cause issues when training my network by introducing false positives, as the model might misclassify intentional design features as printing defects.

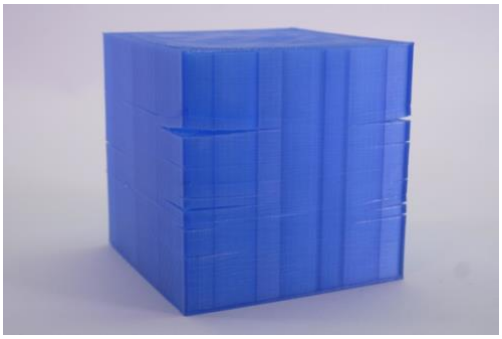

Fig. 4 – Layer separation defect in 3D print (Prusa Research, 2024)

3.2 Existing Additive Manufacturing Fault Detection Systems

It is important to evaluate what systems already exist for additive manufacturing fault detection so I can leverage existing knowledge, address current limitations, and provide a novel or improved solution for accurate fault detection. Here, I investigate what network architectures were used, the datasets used to train the models, their accuracy and evaluation metrics alongside hardware requirements and inference speeds.

3.2.1 Obico (Spaghetti Detective)

Obico, formally known as Spaghetti Detective, is a novel approach to AM fault detection. Instead of training a neural network to classify multiple errors such as warping, layer separation and stringing, Obico's model is trained only to identify Spaghetti errors using a YOLO (you only look once) classification backbone. "*Every time the camera takes a snapshot of the print, the image is sent to TSD backend system, which then run it through the YOLO algorithm*" that "*spits out the coordinates of several boxes with one number associated to each of these boxes to indicate how likely that box contains some kind of spaghetti*" (Jiang, 2019), meaning for each image processed, bounding boxes probabilities of spaghetti can be outputted. Accuracy was evaluated using the F1 metric as it's described to be a balance between precision and recall, as just a high recall can be forged if a model identifies all images as defective and a high precision can be imitated by not alerting on any of the prints.

Their model was trained on a private labelled dataset comprised of spaghetti errors, where bounding boxes were drawn around the defects. Their model achieves recall of 69% and precision of 61% for a F1 score of 65%, catching "*more than 3 out of every 4 print failures, while [it] sends false alarms less than 50% of the times*" (Jiang, 2019). One optimisation found was to make all images greyscale, which should help the model generalise better as spaghetti error look the same independent of colour. In contrast, the research found "*the The mAP [mean average precision] for the model trained with regular color images was 60%, the mAP for the model trained with grayscale images was only 42%*" (Jiang, 2019). As the training set already goes through random augmentations such as colour changes, the additional greyscale conversion may augment samples too far from reality, leading to adverse performance. To prevent having the same issues during training, I should only augment colour or make images greyscale as to not introduce shape variations that could confuse the model.

The official hardware requirements state that edge devices such as Raspberry Pis are not capable of running the model, where more powerful machines such as Jetson Nano 4gb with CUDA acceleration are recommended (Obico, no date b). The requirement of needing CUDA acceleration to run the model on edge effectively (only available on select NVIDIA GPUs), servery limits users who can use the system, requiring non-edge hardware to run if CUDA isn't available. This demonstrates a need for a model which can be inferred on more systems, preferably without needing proprietary enhancements like CUDA which are platform dependant, so the models more accessible.

3.2.2 3D-EDM

3D-EDM (Jeong and Yoo, 2022) is an early detection model for faults in 3D prints utilising a convolutional neural network (CNN) to classify images as defective. The research originally trained their CNN with 10 convolutional layers, reducing the number of layers until reaching an optimised model with 5 convolutions and a fully connected layer for classification. They find that their model can achieve 96.72% test accuracy when performing binary classification and 93.38% when distinguishing between 4 different fault cases (Jeong and Yoo, 2022, p. 2, Table 1). Binary outperforming multi-class classification suggests that higher accuracy can be achieved if my model trains for binary rather than multi-class classification.

The paper explored whether image classification models could be used for fault detection, comparing their model to prior work into fault detection based on processed data such as vibration sensor reading such as work from Bing which uses a support vector machine (SVM) to classify errors with vibrational sensor data (Li et al., 2018). The results concluded that whilst Bing's SVM model achieves accuracy of 94.93%, 3D-EDM outperforms this (Jeong and Yoo, 2022), showing how CNN based models using images without additional sensor data can be proficient at identifying defects.

Whilst the dataset used to train their model is private, the research does reveal that around 1.3k images were selected, where "*200 images were collected for each layer shift, strings, under extrusion, and warping cases*" (Jeong and Yoo, 2022, p. 2) and approximately 500 non-defective images were also obtained from crawling

YouTube and Google. It's also noted that varies transformations and augmentations were applied "*to train the model to effectively recognize the images taken from a different angle*" (Jeong and Yoo, 2022, p. 2). Whilst no comparative results are shown between training the model with and without augmentation, it does seem reasonable that slight, non-disruptive transformations (like rotations, scaling, or flips) would help improve the model's generalization by increasing the diversity of the training data without altering the core features, especially on small datasets such as the one used here. When training my model, I will need to compare results between training with and without augmentation to quantify its impact on performance. This will help determine if the additional data diversity improves the model's ability to generalize and reduces overfitting, particularly given the limited size of the dataset.

3.2.3 Semi-Siamese Network for Robust Change Detection

Niu et al. 2023 pose using a Semi-Siamese neural network to compare a reference image of a 3D print to what is being printed to identity errors. The research predominantly focuses of inkjet 3D printers which deposit droplets of material layer-by-layer, curing them with UV light or fused with powder, in comparison to the standard fused deposition modelling (FDM) printers which extrude heated filament. Their system works by taking a reference image of what the topmost layer should look like (support image) and an image from a camera of the most recent layer. These images get fed into two separate U-Net CNNs which have their own individual weights, sharing the same decoder. Errors can then be identified by generating a change map to localise errors.

Their approach finds that "*defect localization predictions can be made in less than half a second per layer using a standard MacBook Pro while achieving an F1-score of more than 0.9*" (Niu et al. 2023, p. 183). Without revealing which Macbook Pro configuration as used, it's hard to determine the hardware requirements for this model however, since mid 2014 (29th July 2014 Macbook Pro release), all base model Macbook Pros have come standard with at least 8gb of RAM (ROSeaboyer, IAdam1n and InternetArchiveBot, 2024) , and with this paper being published in 2023, it can be reasonably assumed that the device used had a minimum of 8GB RAM. While this indicates the model is efficient enough for consumer-grade laptops, it is unlikely to run in real-time on edge devices like Raspberry Pi, which typically have significantly less processing power and RAM.

When gathering the dataset for training their model, they started "*with only a limited dataset of 57 pairs of experimental images*" (Niu et al. 2023, p. 190), using data augmentation to increase its size, noting how they applied "*zoom, rotation, shear, and position (width and height) shift*" (Niu et al. 2023, p. 190) to expand the dataset.

3.3 Few-Shot Classification Network Architectures

3.3.1 Matching Network

Matching networks (Vinyals et al., 2016) are designed for one and few-shot classification. For each class (way) to classify, a support set of images with the number of images per class specified by its *shot* value, are embedded. To classify a query image, it is first embedded using the feature extractor then, using a weighted distancing algorithm, the distances between the query embedding and class embeddings are evaluated to determine which class the image belongs to.

The network uses a CNN as a feature extractor to provide representations of input images which are then used for matching and classification. It is trained episodically where for each task, a support set and target (query) set are sampled, and the CNN extracts features for all the images. Then, the attention mechanism computes the similarity between each query and support embedding (using cosine similarity), making predictions based on the weighted sum of the support set labels. Gradient decent is typically used to adjust weights of the attention mechanism and embedding functions during training to improve prediction accuracy across episodes.

The network achieves high scores when training and classifying classes with minimal samples per class, such as the Omniglot dataset which contains "*1623 different handwritten characters from 50 writing systems*" (Lake, Salakhutdinov and Tenenbaum, 2015, p. 1333), achieving higher accuracy when classifying between fewer classes and even higher when averaging predictions over 5 attempts (Vinyals et al., 2016, p. 5, Table 1). This shows that the network performs better where there are less classes and more examples per class, likely due to how fewer classes reduce the complexity of the classification task making it clearer what class the query embedding is closest to. More images (shots) per class would also help increase the generalisation of that class which is why performance increases with more shots.

A large drawback for matching networks however is that "*as the support set S grows in size, the computation for each gradient update becomes more expensive*" (Vinyals et al., 2016, p. 8). This is because when measuring how close the query image is to each class, the distance between the query embedding must be measured against every single example in all classes. As noted from evaluation results with the Omniglot dataset, more examples per class (shots) typically result in much higher accuracy. As my neural network would need to run on edge devices in real-time, matching networks require limiting the number of examples per class which may hinder performance.

3.3.2 Prototypical Network

Prototypical networks (Snell et al., 2017) work similarly to matching networks, training episodically using gradient decent, accepting support sets and query images but using squared Euclidean distance calculations to classify the query image. Unlike matching networks, prototypical networks (ProtoNets) do not have an attention mechanism whose weights are updated but instead, during training, the feature extractors weights are adjusted based on the calculated loss, through backpropagation.

In comparison to matching networks, ProtoNets compute mean embeddings for each class, meaning that unlike matching networks which increase in complexity as more examples are added per class, in ProtoNets, despite the number of shots, the query image only needs to be measured against the single mean embedding per class. This makes it better for real-time edge devices because the computation required for each query is reduced to comparing the query embedding with only one prototype per class, rather than computing similarities between multiple support embeddings, which lowers the computational complexity.

Evaluating ProtoNet on the Omniglot dataset, the network achieves strong performance with accuracies 98.8% (1-shot) and 99.7% (5-shot) accuracy in 5-way tasks and 96.0% (1-shot) and 98.9% (5-shot) accuracy in 20-way tasks (Snell et al., 2017, p. 5, Table 1), showing how ProtoNet outperforms the matching network on all metrics when evaluated on the same Omniglot dataset. The paper notes the importance of using squared Euclidean distancing over Cosine, explaining how for 1-shot results, "*since there is only one support point per class, …, matching networks and prototypical networks become equivalent*" (Snell et al., 2017, p. 4), meaning the increased performance seen is mostly down to the choice of distancing.

ProtoNets are also less computationally expensive than other architectures such as matching networks making them more suitable for real-time inference on less powerful edge devices. Unlike matching networks which assign labels based on a weighted sum of support labels using an attention mechanism, ProtoNets assign labels based on the closest class prototype using a much similar squared Euclidean distance calculation, reducing the need for complex computations and making them more efficient for inference.

3.3.3 Comparison

ProtoNets have many advantages over matching networks. For example, to represent a class in a matching network, x number of images based on the number of shots must be embedded and individually stored. When classifying a query image, the distance from its embedding must be individually compared to all the different examples in each classes support set, meaning the more examples you have per class, the slower the model's inference. ProtoNets in comparison however calculate a single embedding for each class by getting the mean embedding of all its support set. This means that despite the number of shots, the ProtoNet will infer at the

same speed. As my model focuses on running on edge devices at real-time speeds accurately, this makes the prototypical network a more suitable choice.

ProtoNets classify via embedding distance calculations rather than dense fully connected layers, which require large matrix multiplications typical in CNNs. This reduces computational complexity and enables faster inference on edge devices. This supports using a few-shot classification model for my defect detection system, as ProtoNets use CNNs as embedding models but omit the fully connected layer, instead producing feature embeddings for distance-based classification. Investigations into fast, edge-deployable CNNs (section 3.4) will highlight efficient architectures, which, once stripped of their fully connected layers, become even more suitable for edge deployment.

3.4 Feature Extractors (Convolution Neural Networks)

Images are complex data structures that machines struggle to interpret. Few-shot neural networks such as the aforementioned matching and prototypical networks therefore require a form of feature extractor to embed images into a known and feature rich form which can be processed effectively. There are a wide variety of convolutional neural networks (CNNs) that can be adapted for feature extraction however it's important to investigate which is most appropriate for real-time, edge inference to ensure my system is not bottlenecked by it.

3.4.1 MobileNet

MobileNets are introduced as a lightweight vision architecture "*for mobile and embedded vision applications*" (Howard et al., 2017, p. 1), focusing on efficiency. To increase inference speed on edge devices, the convolution process is split into depth-wise and point-wise convolutions. In depth-wise convolution, instead of using a typical large filter spanning all channels, separate smaller filters are applied to each input channel, reducing computations, where the point-wise convolution combines the output of depth-wise. The paper also introduces "*two simple global hyper-parameters that efficiently trade off between latency and accuracy*" (Howard et al., 2017, p. 1) with a width multiplier for adjusting the number of channels in each layer and a resolution multiplier, adjusting the input image resolution for enhanced accuracy.

MobileNetV2 (Sandler et al., 2018) improves MobileNet by adding inverted residuals and linear bottlenecks. Instead of shrinking features before depth-wise convolution, it first expands them for better feature extraction, then compresses them back to a small size. Unlike MobileNet, it skips ReLU activation on the final compressed layer to preserve important details. This makes MobileNetV2 more accurate and efficient while keeping it lightweight for mobile and edge devices.

3.4.2 ShuffleNet

Proposed by Zhang et al., 2018, ShuffleNet is "*an extremely computation-efficient CNN architecture … which is designed specially for mobile devices with very limited computing power (e.g. 10-150 MFLOPs)*" (Zhang et al., 2018, p. 1). ShuffleNet uses pointwise group convolution, a 1x1 convolution where input channels are divided into groups and processed separately, reducing the number of calculations and increasing throughput. Then, channel shuffling is introduced which mixes the channels from different groups to ensure the network learns from all the processed groups. This is necessary because, in group convolution, each group processes only a subset of the input channels, potentially limiting the network's ability to learn relationships across all input features. Channel shuffling mitigates this by allowing information to flow between different groups.

ShuffleNetv2 (Ma et al., 2018) builds on this architecture by adding new techniques to specifically enhance inference speed in comparison to floating-point operations a second (FLOPS), such as memory access costs (MAC). In version one, pointwise group convolution requires reading and writing data multiple times so in version two, the feature map channels are just split once in half then merged back together with channel shuffle, reducing MAC and increasing performance. In particular, by splitting the channels and processing only half of them through the convolutions, while keeping the other half as is, ShuffleNetv2 reduces both computational cost and memory access costs, leading to improved efficiency on mobile devices.

3.4.3 Comparison

The data presented in the aforementioned paper (Ma et al., 2018, p. 14, Table 8) compared the performance of several network architectures over classification error and speed on different hardware, using a batch size of 8 for GPU and 1 for ARM based devices. The evaluated models were categorised by their complexity level, defined by their MFLOP requirements, where models such as ShuffleNetv2 are scaled using channel scaling (adjusts the number of channels in each layer) to alter their MFLOP complexity.

This comparison is useful for understanding how the models will run on edge devices such as Raspberry Pi's, which use an ARM based CPU for computations. The table demonstrates that for all complexity levels, ShuffleNetv2 outperforms both MobileNet versions alongside ShuffleNetv1, processing the largest number of images per second on ARM devices, with the lowest error rates for their respective categories. This comes with the exception of MobileNet v1 which seems to process more images a second on the GPU in most categories. It's also important to note that as channel scaling increases for ShuffleNetv2, the error rate is reduced but inference time increases, where at 0.5x scaling the Top-1 error rate is 39.7% with 57 images a second but at 2x the Top-1 error rate is reduced to 25.1%, but performs much slower at 6.7 images a second.

As my project will be implemented on edge devices such as Raspberry Pis which use ARM based processors, ShuffleNetv2 seems to be the most optimal choice of image encoder for my model. This is demonstrated by its fast image processing on ARM devices and low top-1 error rates at all MFLOP complexity levels. Further evaluations into how well the model performs at different channel scales on Raspberry Pi devices will need to be executed to determine the most optimal balance of real-time inference and accuracy.

3.5 Datasets

As we are training a neural network to detect faults in the 3D print domain, we require a labelled dataset of domain specific images showing defective and non-defective prints in varies stages (i.e. during print, finished, and failed). As I want my system to generalise well across many scenarios, it's important that these images showcase a wide variety of camera angles, lighting scenarios and differing models so when training, my model doesn't overfit to specific camera angles and models. Ethically, it is also important that I do not infringe on any copywritten material and have permission to use the dataset, so licences should be reviewed.

One dataset found on Roboflow provides 276 images of print failures "*across a variety of print jobs spanning several types of printer, material, models, and colors*" (QA, 2022). The dataset contains mostly Spaghetti Monster and stringing related print errors, not containing any images of non-failing prints. It is in the public domain, licenced under CC0 1.0 Universal, meaning it can be used without restrictions which is perfect for training my own fault detection model. The variety of camera angles, models and lighting scenarios make this fault related dataset ideal to use in combination with other datasets that contain non-faulty prints for finetuning the feature extractor to generalise across many different prints.

A dataset found on Kaggle (Bhardwaj, 2025), under the MIT licence meaning it can be used without restriction, is a small dataset of 327 total images of finished 3D prints. The images are split into two labelled categories, '*failure*' for prints with defects or '*success*' for prints without faults. A wide variety of material colours, camera angles and lighting scenarios are present throughout the dataset, which is ideal for generalisation.

He, 2022 introduces a collection of 386 labelled 3D print related images in six different categories, '*OK*' of successfully printed prints, alongside five categories for faulty prints: '*blobs*', '*cracks*', '*spaghetti*', '*stringing*' and '*under extrusion*'. Whilst the dataset contains a wide variety of models, camera angles and lighting scenarios, a large proportion of the images contain composited graphics such as text, arrows or circles, artificially inserted during postprocessing. If this dataset was used, these images would have to be removed so the model does not train to recognise overlaying graphics.

Defects of 3D Printing (2024) introduce a moderately sized dataset of 1620 faulty and non-defective prints. The dataset contains 6 labels, one for prints without defects (426) with the rest of the labels representing

different defects such as blobs, cracks, spaghetti, warping and stringing, which for the purpose of training my binary classifier can all be merged into a single label. Different camera angles and lighting settings are used throughout the dataset with certain prints photographed whilst being printed and others outside the printer. Interestingly this dataset contains a lot of images of items printing in situ (not complete) which is useful when training my models with the intended target for inference being actively printing prints.

As my model will be used to recognise faults with 3D prints that are being actively printed, my system should also train on failing and successful prints that are being printed. Incorporating these samples should help my network understand the dynamic changes in the print process and improve its ability to detect faults in real-time. Beyer, 2024 is a dataset hosted on Kaggle containing approximately 12000 images from 34 different print jobs. For each print there are multiple images from a front facing camera positioned in front of the printer, labelled as either '*good*' (5069 images total), '*stringing*' (2798), '*under-extrusion*' (2962) and '*spaghetti*' (134) (Beyer, 2024). The camera angle and lighting scenario stays consistent throughout the images, suggesting that image transformations and augmentations such as horizontal and vertical flips alongside rotations may be advantageous to add more diversity to the dataset, helping the model generalise better.

4 Design

As justified in the research section, I will train a prototypical network using a fine-tuned ShuffleNetv2 model as its feature extractor. This combination was chosen for its efficiency where ProtoNets enable effective few-shot learning with low computational overhead through a mean embedding class system, while ShuffleNetv2 offers high inference speed on ARM-based edge devices due to its architecture minimising memory access costs (MAC), making them most applicable for a detection system on edge-based ARM devices, such as the Raspberry Pi.

4.1 ShuffleNetv2 – Feature Extractor

Layer	Output size	KSize	Stride	Repeat	Output channels			
					0.5×	1×	1.5×	2×
Image	224×224				3	3	3	3
Conv1	112×112	3×3	2	1	24	24	24	24
MaxPool	56×56	3×3	2					
Stage2	28×28		2	1	48	116	176	244
	28×28		1	3				
Stage3	14×14		2	1	96	232	352	488
	14×14		1	7				
Stage4	7×7		2	1	192	464	704	976
	7×7		1	3				
Conv5	7×7	1×1	1	1	1024	1024	1024	2048
GlobalPool	1×1	7×7						
FC					1000	1000	1000	1000
FLOPs					41M	146M	299M	591M
# of Weights					1.4M	2.3M	3.5M	7.4M

Table 1: Overall architecture of ShuffleNet v2, for four different levels of complexities. (Ma et al., 2018, p. 8, Table 5)

4.1.1 Architecture

The ShuffleNetv2's architecture (Ma et al., 2018) focuses on improving inference speeds on ARM based edge devices instead of for improving theoretical metrics such as FLOPS (floating point operations per second). Firstly, the standard model accepts an image that's 224x224 pixels. The image is initially passed through a convolutional layer (conv1) which extracts general features about the image such as edges and textures, producing 24 feature maps and reducing the dimensionality of the image to 112x112. This is fed into a max pooling layer which further down samples the feature maps to 56x56x24.

Then, the feature maps are passed to one of the architectures three main building blocks (stages). "*At the beginning of each unit, the input of c feature channels are split into two branches with c-c' and c' channels, respectively*" (Ma et al. 2018, pp. 8-9), usually a 50/50 split. One half of the split channels remains as an identity branch and passes unchanged through the block, allowing later layers to reuse features from earlier layers directly to improve accuracy. The other half of the split channels go through three convolutional layers, a 1x1 convolutional layer fed into a 3x3 depth wise convolutional layer which extracts features in each layer separately, then a 1x1 convolutional layer which restores the original channel dimensions after the depth wise operation. As depth wise convolutions work on a channel-by-channel basis, they have fewer parameters making them cheaper to run, allowing for efficient processing without sacrificing much spatial feature extraction capability. Batch normalisation and ReLU activation are also introduced after each convolution to stabilize the learning process and introduce non-linearity so the network can learn more complex patterns.

"*After convolution, the two branches are concatenated. So, the number of channels keeps the same*" (Ma et al. 2018, p. 9). By keeping the number of input channels consistent with the output, it avoids needing to use an '*add*' operation which (was needed in the ShuffleNet architecture), improving speed and reducing MAC. The final operation in each stage is channel shuffling where all feature maps in the concatenated stack of identity and transformed channels are mixed, which is done "*to enable information communication between the two branches*" (Ma et al. 2018, p. 9).

After three of these blocks (stages two, three and four in Table 1), a final convolutional layer is "*added right before global averaged pooling to mix up features*" (Ma et al. 2018, p. 9) which helps to combine the learned patterns effectively. Finally, a global average pooling layer averages out each feature map into a single number which is passed to a fully connected layer to make a final prediction. As I am going to be using the model as a

feature extractor inside a prototypical network, the fully connected layer is not necessary as predictions are made based on the distance from the query image to each classes prototype. Instead, using a flattening layer after the global average pooling would convert and output all my features as a single 1D vector which can be passed to the ProtoNet.

The ShuffleNetv2s complexity can also be adjusted based on the choice of which channel scaling to use. Channel scaling alters the number of output channels for convolutions in the stages alongside the final conv5 layer. This can be seen in Table 1 (Ma et al., 2018, p. 8, Table 5), where at 0.5x, stage 2 for example has 48 output channels but at 1x it has 116. I decided to use the 1x scale for my network as it seems to be the best trade-off between accuracy and inference speeds. At 0.5x, the model can achieve a top-1 error rate of 39.7% on ImageNet 2012, at 57 images a second, whilst at 1x it achieves 30.6% top-1 error at 24.4 images a second. Higher multipliers come with diminishing returns, where at 1.5x, the top-1 error rate only improves by 3.2% at the expense of only processing 11.8 images a second (Ma et al., 2018, p. 14, Table 8), more than 2x less image throughout.

4.1.2 Training

As I want to use ShuffleNetv2 as a feature extractor, it's important that it's well adapted to the 3D printing domain. To do this, I will be finetuning ShuffleNetv2, pretrained on a large general dataset such as ImageNet through transfer learning. One study investigating the impact of pretraining using ImageNet found that "*fine-tuning was more accurate than training from random initialization for 189 out of 192 dataset/model combinations*" (Kornblith, Shlens and Le, 2019, p. 2666). The research acknowledges that models trained without pretraining can achieve strong accuracy however training take longer, saying how "*even when fine-tuning and training from scratch achieved similar final accuracy, we could fine-tune the model to this level of accuracy in an order of magnitude fewer steps*" (Kornblith, Shlens and Le, 2019, p. 2667). Due to my limited data and time constraints, I will be fine-tuning a model pretrained on ImageNet.

I will follow the standard ShuffleNetv2 architecture when finetuning, using a fully connected layer as the final output layer for classification. This way, a loss function such as cross-entropy can be utilised to calculate loss and an appropriate optimiser can update the weights, allowing it to learn specific features for classifying defective 3D prints over a labelled dataset. After training is completed, the fully connected layer used for classification can be replaced with a flattening layer to output an array of all features extracted from an image, necessary for integrating the model with the prototypical network.

Overfitting may be encountered during training where the model adapts too closely to the training data leading to poor generalisation but exceptional performance on the training set. It's typically identified when training loss nears zero and accuracy nears 1 whilst the validation performance is weaker. To overcome this, dropout can be introduced, where "*the key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much*" (Srivastava et al., 2014, p. 1). In the ShuffleNet architecture, dropout could be applied to the convolutional layers, after batch normalisation and activation, to help counter overfitting. Typically earlier layers learn general features about images, such as edges or textures and later layers learn more fine-grained details, so lower dropout probability in the earlier layers and a higher probability in the later layers would be beneficial to aid in generalisation by preserving general features.

Freezing earlier layers and only train later layers in the network could prevent overfitting. Research investigating the impact of differing CNN layers found "*first-layer features appear not to be specific to a particular dataset or task, but general in that they are applicable to many datasets and tasks*" (Yosinski et al., 2014, p. 1) meaning typically, the first layers capture general features such as edges and textures. Research by Yosinski et al. (2014) investigates the impact of freezing these earlier, finding "*if the target dataset is small and the number of parameters is large, fine-tuning may result in overfitting, so the [early] features are often left frozen*" (Yosinski et al., 2014, p. 2). As 3D printing datasets are often scarce with few examples per class,

it may be beneficial to freeze earlier layers such as the first convolution (conv1) and the first shuffle block (stage2) as they would have already been pretrained to recognise general features from the ImageNet dataset.

Another approach for fine-tuning is adding and only training additional layers, prior to the final fully connected layer, freezing all the original architecture. Theoretically, as the model is pretrained on ImageNet, it's already well-versed at extracting general features such as textures and edges, as explored by Yosinski et al. (2014). Typically, later layers in a network tend to extract more specific features which is why I will attempt to freeze early layers and only train the added ones. This could mitigate the risk of overfitting on the smaller target dataset while still allowing the model to adapt to domain-specific features as all information learnt during pre-training would be preserved. This approach leverages the general feature extraction capabilities of the pretrained backbone while enabling limited, focused learning through the newly added layers, potentially improving performance without increasing the risk of the model 'forgetting' what it learnt from being trained on ImageNet.

4.1.3 Dataset

As I'm fine-tuning ShuffleNetv2 pretrained on ImageNet, the model will already understand shapes, outlines and general textures meaning fine-tuning must focus on better understanding print specific features relevant to defect detection. Because of this, my dataset should comprise of a wide variety of defective and non-defective prints from varying camera angles, lighting scenarios and filament colours. The objective here is to produce a feature extraction model which understands the nuances of general 3D printed objects to better recognise small imperfections without overfitting to specific prints or camera angles.

As previously investigated, 3D print defect related datasets are uncommon, only containing a few hundred samples per class. For training the feature extractor, the greater numbers of diverse samples per class would help the model generalise more effectively whilst mitigating the risk of overfitting the model, as less samples may lead to faster but less robust convergence. Therefore, for training the feature extractor, I combined multiple datasets to train the model on more samples.

The dataset I will be finetuning ShuffleNetv2 on contains 1668 images, 688 samples of failed prints and 980 successful. It is derived from five different datasets: Projekt (2024), QA (2022), He (2022), Bhardwaj (2025) and Defects of 3D Printing (2024). When curating the dataset, I removed all images where human faces or limbs were visible to protect individual privacy and, to avoid introducing irrelevant features that could negatively impact the feature extractor's performance, I also removed any images where additional graphics such as arrows or text were overlayed on top of the image.

Many of these datasets were pre-augmented, meaning many samples were duplicates of one another with slight adjustments such as colour, brightness or rotation. Data augmentation is useful for artificially adding extra angles and variations to images, typically aiding a model's ability to generalise. However, I will be augmenting images as they are loaded so keeping pre-processed images in my dataset may hinder performance and lead to possible overfitting, as the model could repeatedly see the same augmented sample, possibly learning to remember it. Therefore, I also removed any instances of augmented images, keeping only one original version to ensure diversity and prevent the model from overfitting on redundant variations.

As previously explored, only certain print errors lead to failed prints whilst others, such as stringing, only result in minor defects which can be resolved easily. Due to this, I only included images in my dataset where certain faults are present, such as spaghetti monsters and improper bed adhesion which I identified in my research to be the main faults leading to catastrophic print failures.

4.2 Prototypical Network

4.2.1 Architecture

The prototypical network's architecture comprises of the embedding function which will be the ShuffleNetv2 CNN, where the final fully connected layer used typically for classification will be replaced with a flattening layer to produce a 1D vector of all features (the embedded sample).

As previously discussed, the network compares the similarity (by distance) of an embedded query image to class prototypes to determine the class to which the query image most likely belongs. Prototypes for each class are calculated as the "*mean vector of the embedded support points belonging to its class*" (Snell et al., 2017, p. 2), meaning the set of images representing a class are embedded and their calculated mean is the prototype. Whilst prototypical networks permit any distancing algorithm, the original ProtoNet research found that "*Euclidean distance greatly outperforms the more commonly used cosine similarity*" (Snell et al., 2017, p. 2) and that in particular, "*using squared Euclidean distance can greatly improve results*" (Snell et al., 2017, p. 4).

After measuring the distance between the query and prototype embeddings using squared Euclidean distancing, based on the measured distances between the query embedding and the prototypes, softmax is applied to convert the distances into a probability distribution, and the class with the highest probability (smallest distance) is selected as the prediction result.

4.2.2 Training

Prototypical networks train episodically, "*where each episode is designed to mimic the few-shot task by subsampling classes as well as data points… [making the] training problem more faithful to the test environment and thereby improves generalization*" (Snell et al., 2017, p. 1). For each episode a random subset of classes is sampled from the training set and, for each of these classes, images are sampled for its support and query sets. Then, all the samples are embedded and mean prototypes are calculated for each class, which are used to predict which group the query examples belong to based on their distances from the class prototypes. Loss is then averaged over an episode by "*minimizing the negative log-probability… of the true class*" (Snell et al., 2017, p. 2), where the gradients of the loss function are used by the networks optimiser to update the weights.

Weight decay is a regulation technique to encourage generalisation and prevent overfitting by adding a penalty to large weights during training. It "*suppresses any irrelevant components of the weight vector by choosing the smallest vector that solves the learning problem*" (Krogh and Hertz, 1991, p. 950), meaning the model favours simpler solutions with smaller weights, which helps avoid memorising the training data. This study into weight decay shows that, in non-linear networks such as prototypical networks, "*there is a clear improvement in generalization error when weight decay is used*" (Krogh and Hertz, 1991, p. 956). This is useful for my fault detection model as my dataset is relatively small and it's important that my model can generalise well across many filament colours, printer models, lighting scenarios and structures.

4.2.3 Dataset

As the prototypical network will utilise the fine-tuned feature extractor which has already been trained on a dataset to adapt it to the 3D printing domain, to prevent overfitting it may be beneficial to train the prototypical network using a different dataset. Training over the same dataset may lead to the model memorising characteristics of examples in comparison to learning generalisable features for distinguishing between classes based on their prototypes. I propose using a larger dataset, comprised of a select number of images from the same dataset used to fine-tune the feature extractor alongside samples from Beyer (2024) and Meftahmafazy (2023) (a dataset used to add approximately 300 additional unique samples, unseen during the feature extractors training). As explored previously, the Beyer (2024) dataset is composed of numerous 3D print jobs, each captured from a fixed camera angle, with sequential frames documenting the print process over time and labelled as either defective or non-defective. Training the prototypical network on this dataset is beneficial as it closely mirrors the visual input and conditions the model will encounter in real-world deployment.

As Beyer (2024) only comprises of samples using the same printer, filament colour and fixed camera angle, merging the dataset with that of which was used to fine-tune the feature extractor will hopefully help the model to generalise more proficiently due to the dataset's diversity. To further aid in making the dataset more diverse, simple augmentations can be applied to images such as horizontal and vertical flips to represent differing camera angles. Making slight adjustments to contrast, brightness and saturation may help to introduce slight variations in filament and print colours to aid generalisability however, as explored previously through the research into the Obico model, converting samples to greyscale and performing those augmentations led to their model decreasing in performance in comparison to just converting images to greyscale (Jiang, 2019). This suggests that performing too many augmentations may result in adverse effects, possibly making images steer too far from reality.

The final dataset contains a total of 6112 samples, 3725 images labelled as defective, and 2387 images labelled as successful. I decided to manually sort through the images in the Beyer (2024) dataset as some labelled as defective only contained slight defects such as warping or stringing which, as discussed previously, do not inherently lead to unrecoverable errors. This dataset does not include any augmented or transformed images as I will be performing on-the-fly data augmentation as samples are loaded.

5 Implementation

5.1 Feature Extractor Implementation

Training for the feature extractor was carried out in Python utilising the well-established machine learning library, PyTorch.

5.1.1 Dataset Loading and Augmentation

As previously discussed, I will be training the feature extractor on a dataset consisting of 1668 total samples. When performing transfer learning over a pretrained model, a dataset must be split to produce sets for training, validation and testing. Splitting the dataset ensures that a model's performance can be evaluated effectively, mitigating the chance of overfitting the model to the dataset as the model only trains on a subset of the dataset and evaluates over a differing subset, meaning it can't memorise predictions from the validation set. As my dataset was relatively small, I decided to use a 70/15/15 split, with the largest 70% split being used for training to ensure the model has a substantially diverse number of samples to train on. The dataset's split with a fixed seed meaning all models would be trained using the same samples in each split for consistency, eliminating the possibility that different dataset splits could cause models to perform differently.

As discussed previously, I am to utilise a pretrained ShuffleNetv2 model with standard (1x) channel multipliers, trained on the ImageNet dataset so it's important that my samples match the same dimensions and undergo similar transformations to ensure compatibility with the pretrained model. PyTorch and their subsidiary library TorchVision offer a pretrained ImageNet model through the PyTorch model hub. In the documentation for the model, it explains that images must be at least 224x224 pixels with 3 channels for RGB, "*normalized using mean = [0.485, 0.456, 0.406] and std = [0.229, 0.224, 0.225]*" (Pytorch Team, no date). Normalisation ensures that images are consistent with the data distribution used for ImageNet, which helps the model perform better by aligning the input data with the conditions it was originally trained under. Due to this, I implement a custom data loader which transforms images by normalising them using ImageNet's mean and standard deviation, alongside randomly cropping images to be the required 224x224 pixels.

Certain augmentations where applied to images as they are loaded and sampled. For the training dataset, I created a transformer where each time an image is loaded, it applies slightly different variations of each augmentation. This means the same sample can be augmented differently to how it was last loaded, adding greater variation to the dataset which should help mitigate the risk of overfitting the model as images should always look slightly different. As explored when researching the 3D-EDM project (Jeong and Yoo, 2022), applying random rotations and flips to images can aid generalisation by artificially introducing additional camera angles and positions. When loading images, I have given the transformer a 50% chance to horizontally flip images alongside also applying random rotations to samples between -15 and +15 degrees to artificially add more angles to the dataset.

The training transformer also randomly adjusts the sharpness of an image by a factor of 2 with a 50% probability. This in theory simulates the use of different cameras, as some may be in focus and some may be blurred, which should aid generalisation. Therefore, a small gaussian blur is also applied to samples with a kernel size of 3, meaning all 3x3 pixel areas in an image are averaged whilst maintaining the original image size of 224x224. Applying this small blur to images helps simulate out-of-focus effects, encouraging the model to focus on larger patterns rather than fine details, which should improve robustness and generalisation.

Both the validation and training transformers also convert samples to greyscale whilst maintaining 3 channels. 3D printing filament comes in a wide variety of colours as represented in my dataset across a wide plethora of prints. As print defects appear regardless of filament colour, I don't want my model to learn errors based on colours in images so to mitigate this risk I covert all samples to greyscale which should reduce reliance on colour information and encourage the model to focus on structural and textural features indicative of defects. I keep the number the number of channels as 3 when converting images to greyscale as the pretrained ShuffleNetv2 model expects 3 input channels due to pre-training on ImageNet.

5.1.2 Model Initialisation and Layer Customisation

The ShuffleNetv2 model pretrained on ImageNet (using 1x channel multipliers) was downloaded from the PyTorch model hub which hosts a variety of well-established pretrained model weights (PyTorch Team, no date). Dropout regulation will be used to prevent overfitting. It's typically applied to fully connected layers to randomly remove neurons when training but can also be applied to convolutions to randomly zero a certain percentage of activated features. When a model is seen to be overfitting, dropout can prevent co-adaptation of features in convolutions which is where multiple neurons rely on each other to function correctly leading to overfitting and poor generalisation.

In the ShuffleNetv2 architecture, convolutional layers are followed by batch normalisation then an activation function (ReLU). I add dropout layers after each convolution's activation when training with dropout. The structure Conv, BN, ReLU then Dropout is recommended because placing dropout before batch normalization causes a variance shift, where "*Dropout shifts the variance... [while] BN maintains its statistical variance… [leading to] unstable numerical behavior in inference that leads to erroneous predictions*" (Li et al., 2019, p. 2682). Variance shifts, as explained in the study of dropout and batch normalisation (Li et al., 2019), occur when dropout changes the variance of activations during training, but batch normalisation expects fixed variance at test time leading to unstable behaviour. This is why when I add dropout to convolutions it is applied after activation.

My implementation also has functionality to freeze layers during training to preserve general features learnt in early layers during pretraining, such as edges and textures. As I am training and evaluating multiple models with different configurations, I implemented a way for layers to be specified by name (i.e. conv1, stage2, etc.) and all trainable components in that stage will be frozen (have automatic gradient computations disabled during backpropagation, so their weights aren't updated).

Additionally, I will be evaluating the effects of freezing the original layers in the pretrained network and only training auxiliary layers added prior to the fully connected, classification layer. These new layers will be convolutional as they are meant to be used to learn new features specific to the 3D printing dataset. Since just adding a convolutional layer will lead to linear learning, when adding convolutions, batch normalisation should be attached after to stabilize training and ReLU activation (as used throughout the ShuffleNetv2 architecture) should be inserted to introduce non-linearity so the model can learn more complex patterns. New layers are established in the network by modifying the forward pass logic for the network, redefining the forward function to add the new layers prior to the fully connected layer. Furthermore, a global average pooling layer is introduced between the additional layers and fully connected binary classification layer to reduce spatial dimensions and generate a fixed-size feature vector for classification regardless of an input image's dimensions.

5.1.3 Training

During training I focus on improving the model's accuracy by freezing layers, implementing regulation such as dropout and amending how the dataset is loaded. Due to time constraints, I keep the optimiser and loss criterion the same across all models when training, ensuring consistency in evaluation and comparison of performance while prioritising architecture adjustments and dataset handling for optimisation. I will be using the standard cross-entropy loss function during backpropagation, as it is widely used in CNN training for classification tasks. It is also imperative to choose an optimizer that works generally well across different models, ensuring stable and reliable performance.

The original ShuffleNetv2 paper references using the stochastic gradient decent (SGD) optimiser when training and evaluating (Ma et al., 2018) as proposed and used in the original ShuffleNet evaluations (Zhang et al., 2018) which were obtained from Xie et al. (2017). Other advances and research into differing parameters for the ShuffleNetv2 architecture suggest that the Adam (Adaptive Moment Estimation) optimiser may be a more applicable choice.

Research into different optimisers used to train models for classification found using the Adam over SGD on small datasets achieves lower loss and higher accuracy. Their findings found on a dataset with 1885 total samples and 8 classes, ShuffleNetv2 benefits from using Adam over SGD with an increased accuracy of 0.148 and a decreased loss of 1.0929 (Sun et al., 2024, p. 9, Table 4). Furthermore, when evaluating on a binary classification dataset with 8621 samples, using Adam over SGD yields a 0.1981 increase in accuracy and an acceptable marginal decrease of 0.0268 for loss (Sun et al., 2024, p. 11, Table 5).

Another investigation studying the impact of optimisers and learning rate on the performance of deep learning-based algorithms for rock thin-section image classification on a small dataset with 2634 total images over 3 classes (Li, Zhao and Ma, 2022, p. 5, Table 1) found using Adam over SGD in ShuffleNetv2 offered increases in prediction accuracies across all classes, with increases of 20%, 7% and 7% for each classes respective F1-scores (Li, Zhao and Ma, 2022, p. 21, Table 8). Due to the evident increase in accuracy when using Adam over SGD, I will be utilising Adam as the optimiser for training my models.

The model is trained through transfer learning where for each epoch, batches of images (based on a specified batch size) are sampled from the training dataset and transformed using the training transformer. The data and labels are then passed forward through the model to make predictions. The cross-entropy loss criterion then computes the loss of those predictions by backpropagating to compute gradients which are used by the Adam optimiser to update the model's parameters effectively. After each epoch is completed, the validation set is passed to the frozen model, and the criterion is used to evaluate the model's current performance on the untrained validation set. This gives an overview of how the model performs on data that it was not trained on, which when compared to the test dataset's accuracy and loss can help inform whether the model is overfitting or generalising well.

5.2 Few-Shot Neural Network Implementation

To train the prototypical network, I leveraged an existing GitHub repository (Snell and Whitcomb, 2018), created by Jake Snell, a co-author of the original network's paper (Snell et al., 2017). The repository contains code to load, train and evaluate the network on the Omniglot dataset, so heavy modifications had to be made so the model can load and train on my dataset. The encoder logic also had to be updated to allow the trained ShuffleNetv2 models to be loaded and used as encoders for the network.

5.2.1 Dataset Loading and Augmentation

My dataset's file structure differs significantly from that used in the Omniglot dataset so custom code was implemented to load it. Similarly to when loading the dataset for training the feature extractor, a 70/15/15 dataset split for training, validating and testing was used with a fixed seed to ensure the same images are sampled each time a new model is trained, establishing consistency and comparability across training runs. Furthermore, the same augmentations and transformations were applied to their respective dataset splits as used for training the feature extractor. Using identical preprocessing steps helps maintain consistency in data distribution (i.e. using the same normalisation as required by ImageNet) and ensures fair evaluation of model performance. To aid in performance, images are cached locally after being loaded and transformed, reducing read times and ensuring faster data retrieval during training and evaluation.

As Prototypical networks train episodically, the dataset loader must be able to sample images based on a given a number of support and query samples to extract for each class. I do this by randomly shuffling the images from a class and splitting them into a support set of n support images and a query set of n query images. This ensures that each episode contains balanced and varied examples, allowing the model to generalise better by learning class prototypes from limited support samples and evaluating performance on unseen query samples in each episode.

5.2.2 Model Initialisation and Layer Customisation

The original training implementation in the repository utilised a novel CNN containing 4 linear blocks of convolutions with batch normalisation, ReLU activation and max pooling, all feeding into a final flattening

block to produce fixed-length feature embeddings for each input image, which are then used to compute class prototypes and perform distance-based classification in the embedding space. As I'm evaluating the performance of the network with different variants of the ShuffleNetv2 CNN, I need a custom implementation to integrate them.

For evaluating the performance of training the network with a ShuffleNetv2 encoder using the standard architecture, I load the trained models directly from their weights into the same PyTorch ShuffleNetv2 class used during fine-tuning. For ShuffleNetv2 to work with this network, it must output a representation of the features. After loading the weights of the model, I set the fully connected layer of the network to be an identity layer, meaning the input passes right to the output unchanged, which then feeds into a flattening layer which converts the feature maps the network produces finto a 1D feature vector, representing the embedding of the passed image. This embedding is what is then used for calculating prototypes and measuring the distance between the query image and class prototypes.

I also evaluate the effectiveness of training the feature extractor with additional layers and dropout. To load models with additional layers not present in the original architecture, a new class inheriting from ShuffleNetv2 is created to override the forward pass function of the network to implement the additional layers. Furthermore, before weights are loaded for models, dropout blocks are inserted where described in the model's configuration (i.e. in stage2, stage3, etc.). This modular approach to loading models allows dynamic evaluations of all trained ShuffleNetv2 variants to help assess their performance under different configurations more easily.

5.2.3 Training

The training code for the prototypical network remains mostly unmodified from the original implementation (Snell and Whitcomb, 2018). Within each epoch, the dataset is episodically sampled. For each batch of samples in an individual episode, loss is first calculated. The calculated loss is then backpropagated through the network to compute the gradients for how each model parameter contributed to the loss, where those gradients are passed to the optimiser which uses them to update the model's weights before resetting the gradients for the next batch. Various hooks are also used throughout the training process, such as if weight decay is specified, at the start of each epoch the learning rate can be decreased in proportion to the given decaying rate.

Loss is evaluated by encoding the support and query images using the feature extractor, then obtaining class prototypes by computing the mean embedding of each classes support sets. Euclidean distancing is then used to compute distances between each query embedding and class prototypes. As stated in the paper (Snell et al., 2017), log-softmax is then applied to their negative distances to convert the distances into log-probabilities, where smaller distances have higher probabilities, so closer embeddings should represent greater similarity between classes. These log-probabilities are then used to compute the negative log-likelihood of the true labels by averaging the log-probabilities assigned to the correct class across queries. PyTorch's implamentation of cross-entropy loss notes that it "*is equivalent to applying LogSoftmax on an input, followed by NLLLoss*" (PyTorch Contributors, 2024), meaning when calculating loss for our network, we are doing the equivalent to cross-entropy loss, with negative Euclidean distances acting as the logits with the main difference being that prototypes are computed dynamically from the support set, rather than being fixed.

5.3 Evaluation Metrics

For evaluating the performance of both the feature extractor when finetuning and the prototypical network when training, I calculate and store both the model's training and validation set's loss and accuracy at each epoch. Loss measures how far predictions are from true labels, representing the average percentage of misclassifications, whilst accuracy evaluates the proportion of predictions made that were correct as a percentage. These values are useful for training and adjusting model parameters as they can assist in assessing if models are overfitting (typically a high training accuracy with a lower validation accuracy represents a model remembering the training dataset, a form of overfitting), underfitting (high losses and low accuracies in both

validation and training datasets) or generalising well (similar accuracies and losses across both the training and validation sets).

In the prototypical network, for each batch in an epoch the training dataset's loss and accuracy are calculated and appended to a running average which, at the end of each epoch calculates the average evaluation metrics. Performance on the validation dataset is evaluated at the end of each epoch where both the train and validation evaluation metrics are recorded. Similarly, when training the feature extractor, for each batch used in training, the loss from the criterion is tracked and used to update a running loss for the entire epoch, alongside a running accuracy. After an epoch finishes training, the validation set is evaluated to obtain the model's validation accuracy and loss at each epoch. All metrics are stored so that they can be analysed and plotted for trend analysis and model performance evaluation across epochs.

6 Testing and Evaluation

6.1 Feature Extractor Training and Evaluation

6.1.1 Dropout and Layer Freezing

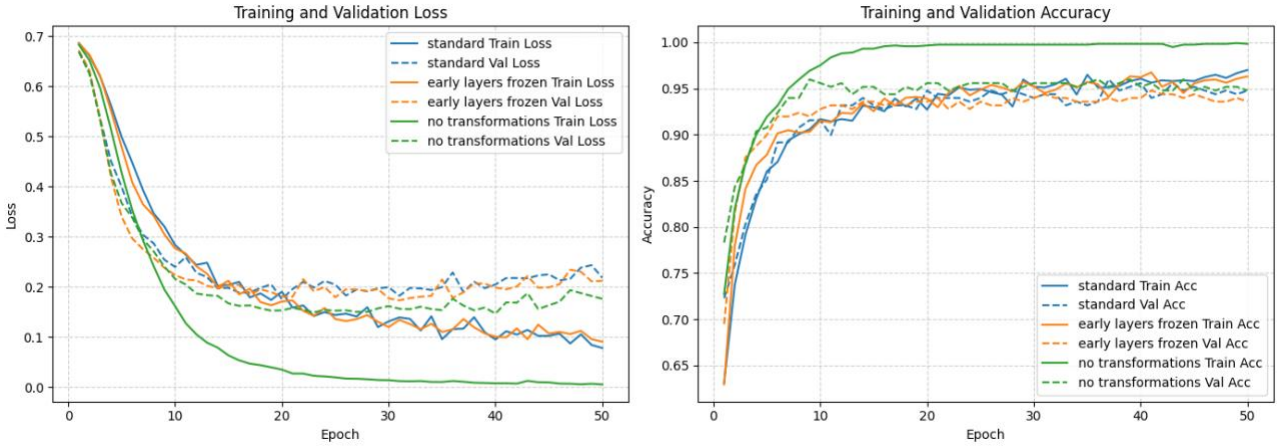

Fig. 5 - Plots comparing evaluations metrics of standard ShuffleNetv2 model training, with and without layer freezing and dropout. Numerical results in Table A1.

As discussed previously, all models are trained with cross-entropy loss, utilising the Adam optimiser. I first trained using a learning rate of 1e-4, without any dropout applied to the model so that I could evaluate the effectiveness of transformations and augmentations to the dataset (Fig. 5). Without performing transformations such as normalisation, the model underfits to the training dataset, converging early before severely overfitting (Fig. 5). At the 8th epoch before overfitting, the disparity between training and validation loss was -0.065 (Table A1) indicating that the model had not yet learned sufficiently and was underfitting the training data.

Applying transformations to the dataset improved accuracy and prevented premature overfitting, allowing for longer training. The best evaluation metrics were recorded at the 19th epoch (Table A1), where training for longer results in similar accuracies but severe divergence in losses between the validation and training datasets, where the training loss improves to 0.078 in comparison to validation loss at 0.218 (Fig. 5). This suggests that data transformations improve generalisation, reducing overfitting, but extended training can still lead to loss divergence, indicating that while accuracy plateaus, the model continues to overfit in terms of loss. This is also evident when training with the first two early layers of the network (conv1 and stage2) frozen and transforming the dataset, which was able to train for longer before overfitting past epoch 30. This indicates that whilst performing transformations helps prevent early convergence, the models struggle with overfitting during extended training, suggesting the need for additional regularisation methods such as dropout.

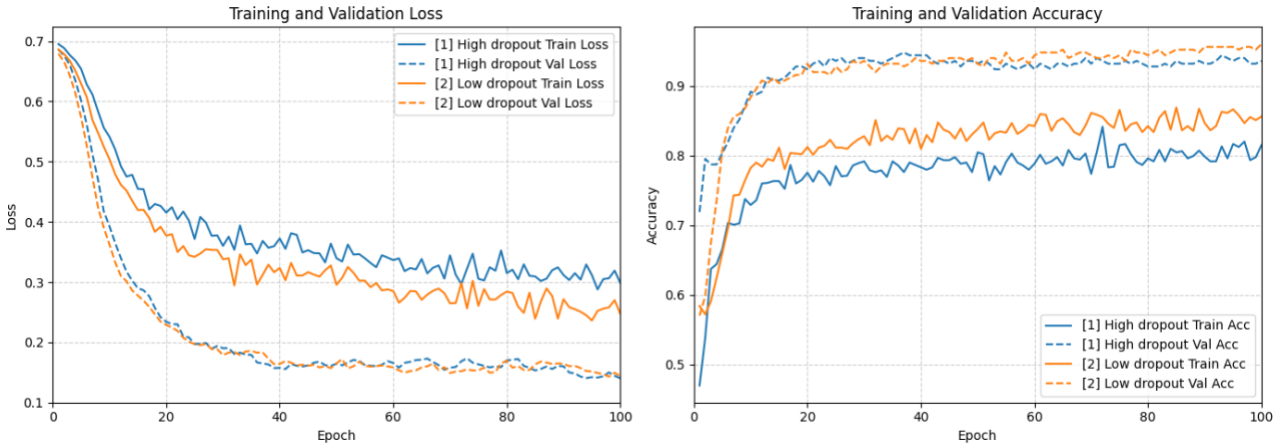

Fig. 6 - Plots comparing the evaluations of ShuffleNetv2 models with different dropout configurations over 100 epochs. Results in Table A2.

To mitigate the risk of overfitting the model, I evaluated the effect of regulating using different dropout rates in varying layers. As freezing the earlier layers of model's did not seem to improve the model's performance so far (Fig. 5), we kept all layers unfrozen whilst also performing all transformations as that prevented the models in Fig. 5 from converging too quickly. I first applied high dropout rates to the later layers of the network, with a rate of 50% in the fully connected and conv5 layers, while applying lower dropout rates of 2.5% and 5% to stages 3 and 4, respectively (Fig. 6). This prevented premature convergence, allowing the model to train for 100 epochs however also led to greater levels of underfitting (Table A2) where validation metrics outperform the training sets, possibly due to the application of strong regulation in deeper layers preventing learning of complex patterns.

Lowering the dropout in the fully connected and conv5 layers to 0.4 and 0.3 respectively still prevented early convergence whilst also allowing the model to learn more complex patterns in comparison to using the higher dropout rates (Fig. 6). This improvement indicates that previously the dropout probabilities were too high in later layers, making it difficult for the model to learn complex patterns. The original dropout paper evaluated the impact of dropout on differing dataset sizes, finding that "*it can be observed that for extremely small data sets (100, 500) dropout does not give any improvements*" noting that "*there is a 'sweet spot' corresponding to some amount of data that is large enough to not be memorized in spite of the noise but not so large that overfitting is not a problem anyways*" (Srivastava et al., 2014, p. 1946). The dataset being used to fine-tune this model has 1668 total samples making is a relatively small dataset, suggesting that large rates of dropout may hinder performance as seen in Srivastava et al. (2014), suggesting smaller rates of dropout would aid prevent both underfitting (as seen in Fig. 6) and overfitting (as seen in Fig. 5).

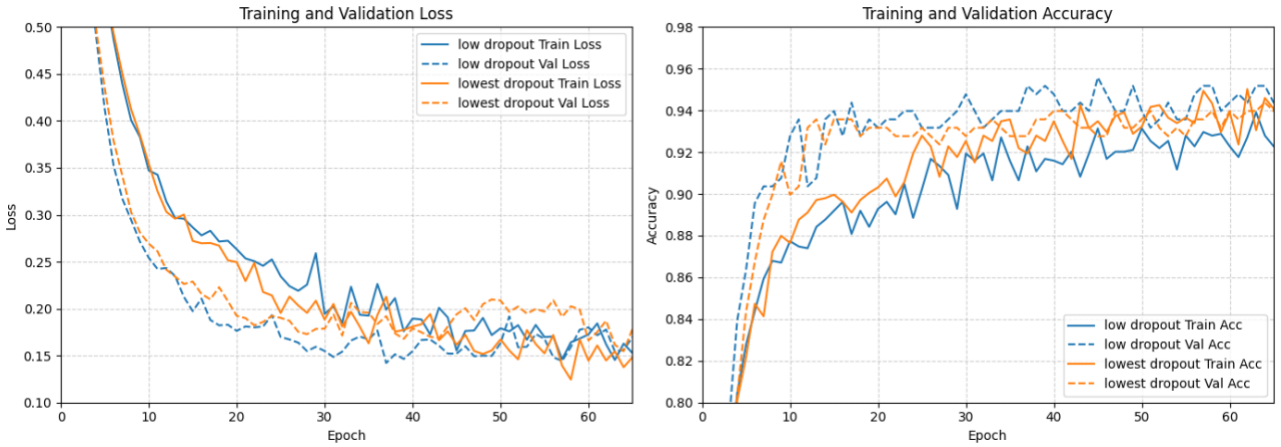

Fig. 7 – Plots comparing the evaluations of ShuffleNetv2 models with differing dropout configurations with low dropout probability rates over 65 epochs. Numerical results in Table A3.

Following this finding, I then evaluated the performance of the models with lower dropout probabilities to assess if my assumption that higher dropout rates on small datasets can cause underfitting. Using the same parameters as before but with dropout rates reduced in the fully connected and conv5 layers to 20% and 15% with 1% in both stages 3 and 4, it can be observed that the issue of underfitting seems less prominent as both the training and validation evaluation results converge around the same point (Fig. 7). Reducing the dropout rates further in the fully connected and conv5 layers to 10% increased performance even more, achieving the best fine-tuned model's performance of 93.1% and 94% accuracies with losses of 0.155 and 0.162 over the training and validation sets respectively (Table A3). This shows that a small amount of dropout aids in helping reduce overfitting (in Fig. 5), but excessive use of it with high dropout probabilities (Fig. 6) can lead to model's severely underfitting, unable to learn complex patterns in the dataset.

6.1.2 Additional Layers

I also evaluated the effectiveness of adding and only training additional convolutions near the final fully connected layer, freezing the other layers. As the model is already pretrained on the ImageNet dataset, making

it well suited for general feature extraction, freezing its layers may help preserve what is learnt, mitigating the risk that training those layers could override crucial learnt patterns.

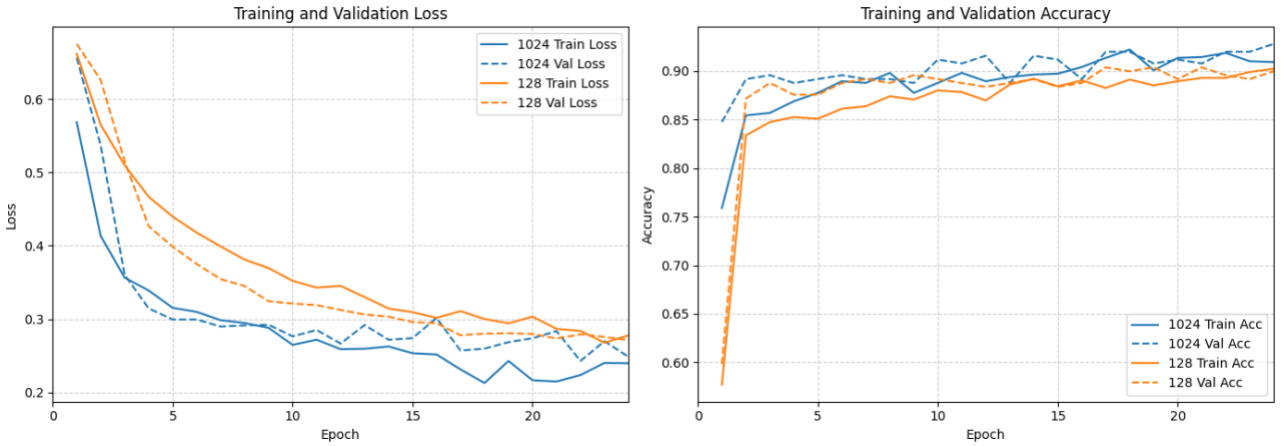

Fig. 8 – Plot comparing the validation and training loss and accuracies for ShuffleNetv2 models frozen, where only additional convolutions are trainable. Numerical results in Table A4.

I first evaluate the effect of training only a single additional convolution using differing numbers of output channels. I use a learning rate of 1e-4, no dropout and batch size of 64, applying transformations to the dataset. The ShuffleNetv2 architecture outputs 1024 channels from its final convolutional layer (conv5) (Ma et al., 2018, p. 8, Table 5). To preserve learnt detail, I first evaluate the performance of a model with an additional convolution with 1024 output channels (alongside batch normalisation and ReLU activation as previously stated). The model demonstrates good generalization, with high and closely matching training and validation accuracy, and low, comparable loss values with a slight discrepancy between the training and validation losses which could indicate slight overfitting.

While these results seem adequate, adding an additional convolution with 1024 channels can be computationally expensive. Studies evaluating the computational cost of the number of filters (directly influenced by the number of output channels) find that the cost of a standard convolutional layer increases linearly as the number of output channels increase (Denton et al., 2014). As the final network I am training for defect detection is targeted towards edge devices, it's important I keep complexity down. Reducing the output channels from 1024 to 128 shows comparable convergence with a slight reduction in accuracy, decreasing validation accuracy by 0.024 and increasing loss by 0.038 (Table A4). This implies that fewer filters can be used, reducing the model's complexity whilst only losing a marginal amount of precision.

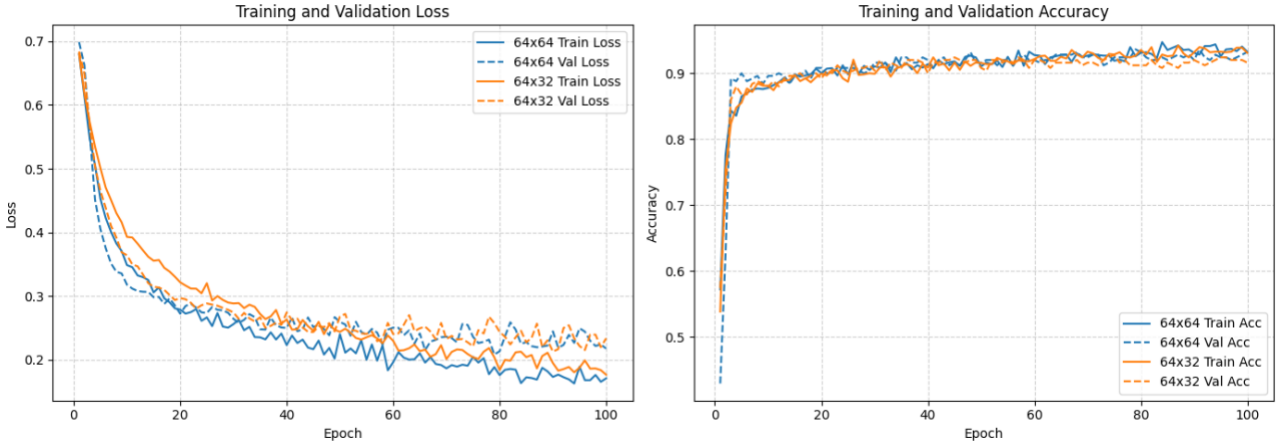

Fig. 9 – Plots comparing the training and validation loss and accuracies of ShuffleNetv2 models training only additional convolutions over 100 epochs. Numerical results and model configurations in Table A5.

Noticing that decreasing the filters in convolutions only marginally reduces precision, I evaluated the effectiveness of using multiple, smaller connected convolutions with interlaced activations and batch normalisation, as previously stated. Using a single convolution with reduced filters, and activation to introduce non-linearity, may not be large enough to capture all relations in the dataset. Therefore, introducing more convolutions (with their own activations and normalisations) may enable the model to capture more in-depth details and relationships between features. As more convolutions are being used, I further reduce the number of output channels as training a small dataset on large convolutions risks overfitting the model.

Using the same parameters as before, changing the additionally added convolutions to use a convolution with 64 output channels followed by activation, batch normalisation then another 64-channel convolution (and batch normalisation and activation), we find this model is a significant improvement over using a single, 128-channel convolution (Fig. 9), with improvements to all evaluation metrics (Table A5). This could be due to how utilising activation between the two trainable convolutions introduces non-linearity helps the model learn more complex features, as explored in reviews of deep learning, exploring how "*non-linear performance of the activation layers means that the mapping of input to output will be non-linear; moreover, these layers give the CNN the ability to learn extra-complicated things.*" (Alzubaidi et al., 2021, p.18). Furthermore, whilst the validation and training accuracies seem similar, the losses between sets display a small disparity, where the training loss is 0.0467 lower (Table A5), implying that the model could be slightly overfitting to the training set.

As the increased number of convolutions seemed to aid performance, I also evaluated the effect of reducing the output channels to reduce the model's complexity for faster inference on edge devices with minimal compute. Changing the number of output channels in the second additional convolution from 64 to 32, I find that the model performs worse, with greater disparity between the training and validations sets (where the model performs better on the validation set) (Table A5), indicating that reducing the output channels worsened the overfitting issue. This could imply that the model is not large enough to capture relations needed for generalisation, leading to underfitting and poorer overall performance.

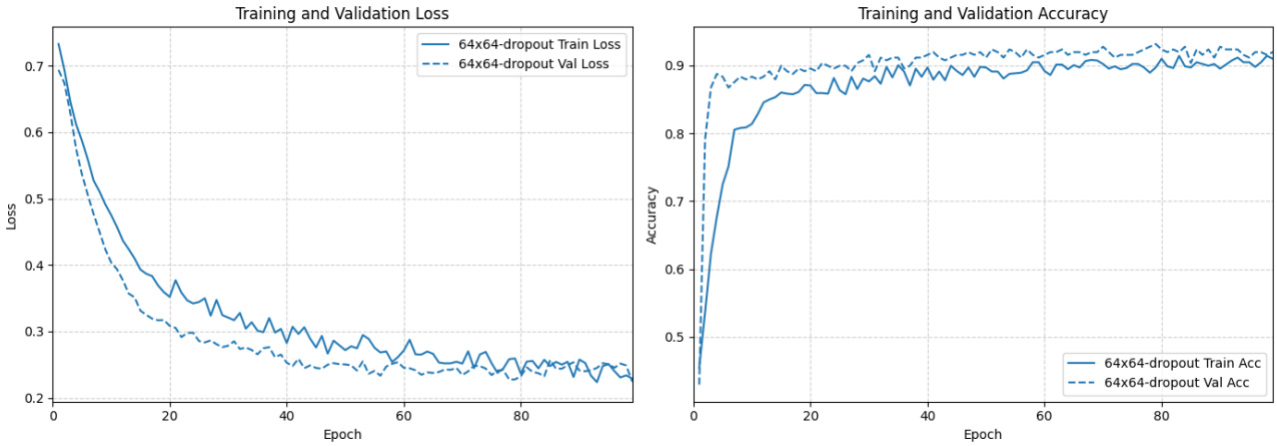

Fig. 10 – Plots comparing the training and validation loss and accuracies of a ShuffleNetv2 model trained over 100 epochs. Numerical results and model configuration in Table A6.

Due to this overfitting, I evaluated implementing dropout after each activation in the trainable additional layers. I found a dropout rate of 40% resulted in the least overfitting before models started underfitting. Whilst applying dropout regulation allowed the model to generalise better, it also resulted in decreased performance, with higher average losses and lower accuracies that without dropout regulation (Table A6). This trade-off may occur because dropout reduces overfitting by restricting the models ability to fully learn complex patterns in the data.

6.1.3 Model Selection

The most performant ShuffleNetv2 model fine-tuned for defect detection achieves 93.1% train and 94% validation accuracy with 0.155 train and 0.162 validation loss (Table A3). Using no additional layers, the model

was trained with conv1 and stage2 frozen, using a small learning rate of 1e-4 and 10% dropout applied to the fully connected and conv5 layers, alongside 1% in stages 3 and 4. The dataset is augmented and transformed, using a batch size of 64. From the evaluations I find that training just additional convolutions appended to the model achieves worse performance than freezing early layers and only training later layers alongside also worsening the computational complexity of the model. For this reason, due to the good generalisation and appropriate performance of this feature extractor, I will use it in the prototypical network.

6.2 Few-shot Network Training and Evaluation

To evaluate the effectiveness of finetuning the feature extractor prior to training the prototypical network, I will be training and examining models using both the standard ShuffleNetv2 model, pretrained on the ImageNet dataset alongside the most appropriate fine-tuned model, as outlined in section 6.1.3. In section 4.1.3 I hypothesised that utilising a combination of samples from feature extractor dataset and sets of unseen samples (Beyer, 2024 and Meftahmafazy, 2023) may lead to the best results and least overfitting.

Evaluating the performance of models on only the validation set of the dataset will provide good insights into that datasets accuracy, but to properly assess generalisation, it is essential to test on additional, unseen datasets hence, I evaluate the models across multiple 3D fault-related datasets. There are four total, set 1, aptly named '*prototypical network dataset*' is as described in section 4.2.3 and set 2 is the dataset used to fine-tune the feature extractor, named '*CNN Dataset*'. As prototypical networks "*reflect a simpler inductive bias that is beneficial in [a] limited-data regime*" (Snell et al., 2017, p. 1), dataset 3 ('*prototypical network dataset (reduced)*') comprises only of select examples from set 1, reducing its total size to 189 which enables evaluation of model performance under few-shot conditions. Finally, dataset 4 named '*Beyer, 2024 Dataset (unfiltered)*' contains all available samples from the original collection so that models can be evaluated on unseen samples of in situ prints being manufactured.

6.2.1 Fine-tuned Feature Extractor

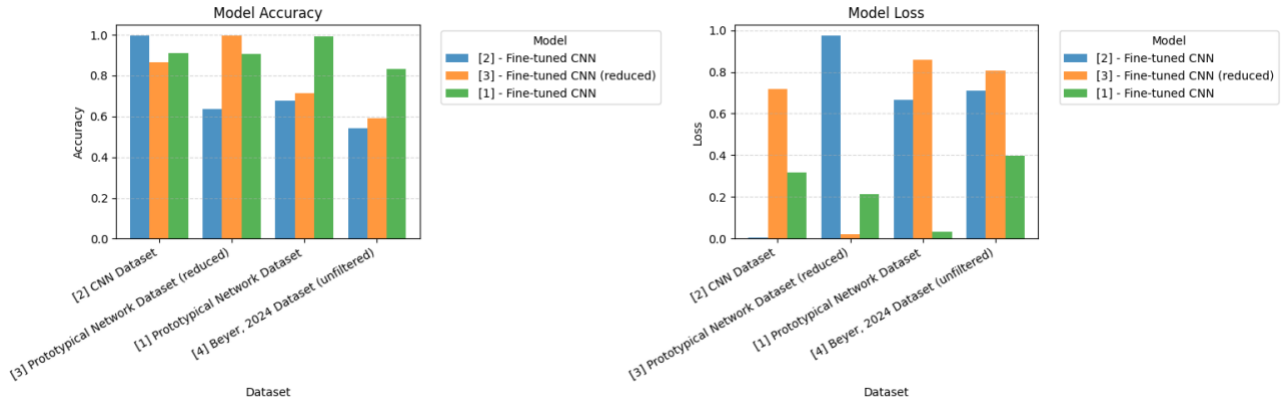

Fig. 11 – Grouped bar chart of prototypical network models using the fine-tuned feature extractor, showing their accuracies (left) and losses (right) over all different datasets. Numerical results in Table B1.

Evaluating optimised models trained on datasets 1 through 3, utilising the most proficient fine-tuned feature extractor (as identified in section 6.1.3), it's evident that training using dataset 1 (the dataset curated for training the prototypical network) results in the best performance (Fig. 11), performing best on the prototypical dataset 1 and worse on set 4, Beyer (2024). All other models trained using differing datasets were found to perform extremely adversely, all achieving more than 0.5 loss on datasets other than the ones used to train them. The adverse performance on dataset 4 could be due to how, in an unfiltered state, that dataset contains samples in the defective class very similar to non-defective prints, leading to more misclassifications, reflecting how using a mix of samples from the feature extractor training dataset alongside new, unseen samples results in the best generalisation, as hypothesised.

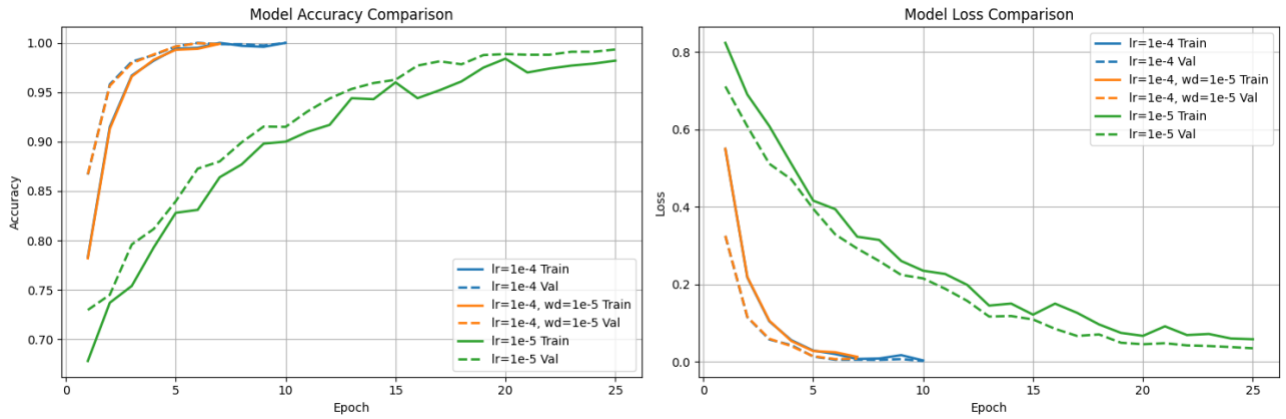

Fig. 12 – Accuracies (left) and loss (right) evaluation plots for prototypical networks trained on dataset one over 50 epochs. Numerical results in Table B2.

To achieve the results of the most performant model in Fig. 11, different parameters were evaluated during training. All models used the standard prototypical network configuration of 5 samples per prototype, evaluated over 5 examples during episodes and 15 during validation, over 100 episodes per epoch. Training using an initial low learning rate of 1e-4 resulted in the worse performance, overfitting after 10 epochs reaching accuracies on both validation and train sets of 100%, with losses of 0.003 and 0.001 on the train and validation sets respectively (Fig. 12).

As previously explored, weight decay can be used to regulate overfitting by encouraging smaller weights by adding penalties to large ones. Applying a weight decay of 1e-5, as suggested in the original network's paper (Snell, et al., 2017, p. 7), it's apparent that the model still overfits, converging at the 7th epoch with a loss near 0 and accuracies near 100% (Table B2). Weight decay not helping reduce overfitting could be a sign that the learning rate is too high, resulting in the model overshooting optimal weights and failing to generalise effectively.

Achieving 100% accuracy indicates that the model is memorising data, most likely due to having too high of a learning rate, resulting in rapid convergence without generalisation. This is justified by studies into the Adam optimiser, where it can be "*observed to generalize poorly compared with SGD or even fail to converge due to unstable and extreme learning rates*" (Luo et al., 2019, p. 1). Lowering the learning rate by a factor of 10 (without weight decay), results in the most optimal model as described previously (Fig. 12), converging after 25 epochs.

6.2.2 Standard Feature Extractor

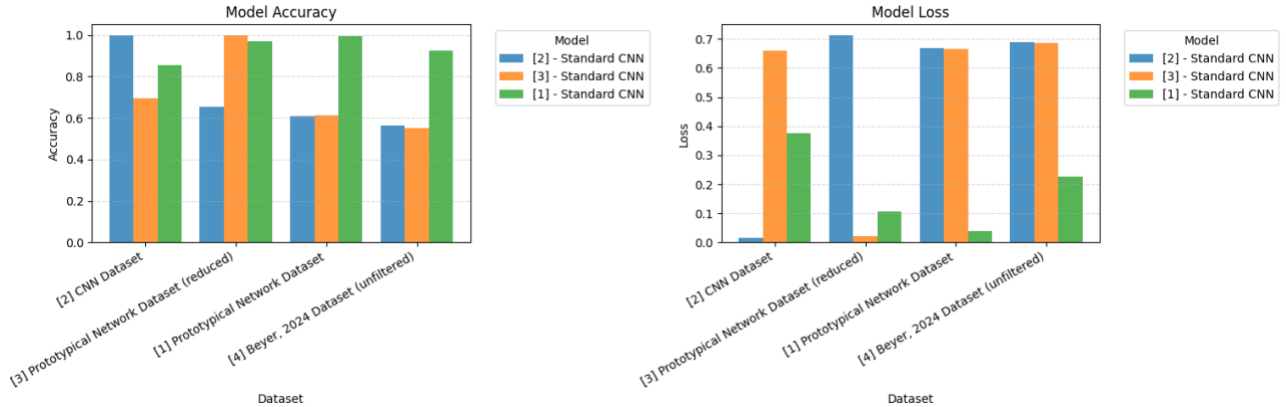

Fig. 13 – Grouped bar chart showing accuracy (right) and losses (left) of prototypical network models trained using a non-fine-tuned feature extractor, pretrained on ImageNet. Numerical results in Table B3.

Training the network using the non-fine-tuned ShuffleNetv2 feature extractor, pretrained on the ImageNet dataset, results in the most performant model with the best generalisation in comparison to using the fine-tuned

CNN. The best model was trained using the designated dataset for the prototypical network (dataset 1), achieving accuracies above 85% and losses below 0.38 across all datasets (Table B3). This is significantly more performant than before, with average accuracies of 93.6% and losses of 0.187 (Table 2), in comparison to the best fine-tuned model achieving averages of 0.911 accuracy and 0.242 loss (Fig. 11).

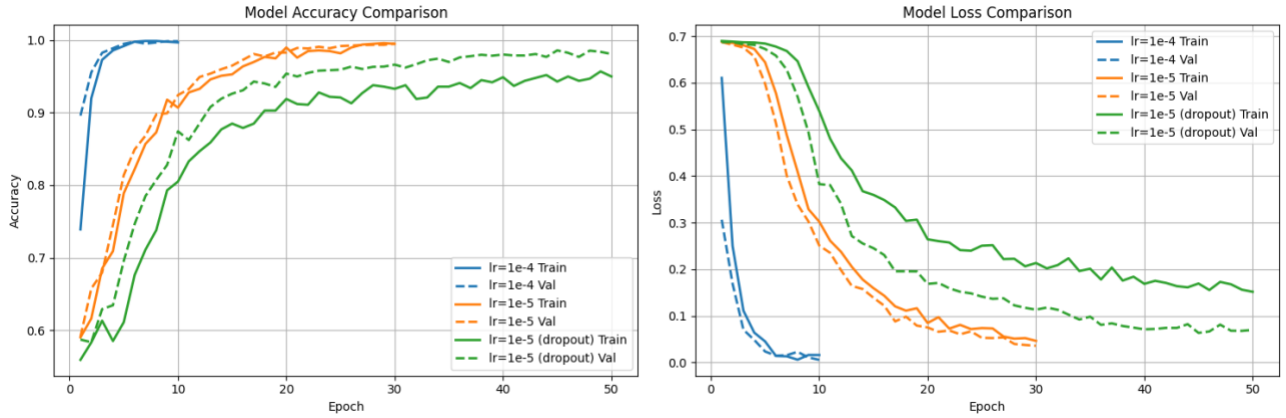

Fig. 14 – Validation and training set evaluations for prototypical network models using the non-fine-tuned feature extractor over 50 epochs. Plotting the model's accuracy (left) and loss (right). Numerical results in Table B4.

To arrive at this model, I evaluated the performance with varying parameter configurations. Using a learning rate of 1e-4, as recommended in the original network's paper (Snell et al., 2017, p. 7) but without weight decay, the model is seen to converge quickly within 10 epochs whilst overfitting to the dataset (Fig. 14). As explored previously, this signals that the model converges too quickly and overshoots the optimal solution, memorising the dataset instead of learning the underlying patterns, causing poor generalisation due to a high learning rate.

To counter this, I found dropping the learning rate to 1e-5 prevent premature convergences, arriving at 99.5% accuracy with 0.05 and 0.04 loss across the training and validation sets respectively. Further models were trained and evaluated using weight decay and dropout as an attempt to see if the model could be improved (see Fig. B5) however this was the most optimal model configuration.

6.2.3 Model Selection

Unexpectedly, the best model was trained using the non-fine-tuned feature extractor, pretrained on the ImageNet dataset. It was trained over 30 epochs with 100 episodes per epoch, using a learning rate of 1e-5 on dataset 1, the dataset for training the prototypical network (section 4.2.3). The network used 5 samples per prototype and 5 samples to evaluate loss during training, transforming and augmenting the dataset as explained in section 5.2.1. The model scored 99.5% accuracy across the training and validation sets with losses of 0.047 and 0.036 respectively (Table B4), showing good signs of generalisation, which is reinforced by the evaluations showing adequate generalisation over multiple datasets. (Fig. 13).

The non-fine-tuned feature extractor achieving better results could be a consequence of how meta learning (the training strategy utilised by prototypical networks) differs from transfer learning (method used to train the ShuffleNetv2 feature extractor). Due to the small size of the dataset used to fine-tune the feature extractor, the training may have resulted in overly specialising features to that specific dataset, potentially losing robust, general patterns learnt from ImageNet. This meant that during meta-learning when training on unique unseen samples, the model may have struggled to generalise effectively due to the lack of diverse, transferable features. A study into the effectiveness of learned embeddings versus complex meta-learning algorithms for few-shot image classification reinforces this idea, finding "*standard cross-entropy pre-training is sufficient to generate strong embeddings without meta-learning techniques or any fine-tuning*" (Tian et al., 2020, p. 6).

6.3 Edge Device Evaluation

Model throughputs were evaluated on a Raspberry Pi 4B with 4GB of RAM (2018) using a clean install of Raspberry Pi OS Lite (released 28/10/2024) built on Debian bookworm version 12. As Obico's Spaghetti

Detective (Obico, no date a) is the only other publicly accessible FDM fault detection model, I also evaluated it's performance using a modified version of its inference code (Obico, 2025). Both models were assessed on the test subset of each dataset which was not seen during training, used to produce an accuracy (percentage of correct predictions), loss (how wrong predictions are) alongside a calculated F1 score (a metric used to represent the precision and recall of the model over a dataset, where a higher score generally indicates better performance).

Dataset	**Accuracy**	**Loss**	**F1 Score**	**Throughput (images/sec)**
Prototypical Network (Ours)				
[1] Prototypical Network Dataset	99.5%	0.040	0.995	-
[2] CNN Dataset	85.5%	0.376	0.860	-
[3] Prototypical Network Dataset (reduced)	97.1%	0.106	0.971	-
[4] Beyer, 2024 Dataset (unfiltered)	92.4%	0.226	0.923	-
Average	**93.6%**	**0.187**	**0.937**	**15.10**
Obico Spaghetti Detective (Obico, no date a)				
[1] Prototypical Network Dataset	57.03%	10.8280	0.5183	-
[2] CNN Dataset	84.0%6	10.3906	0.7674	-
[3] Prototypical Network Dataset (reduced)	79.31%	11.1010	0.6667	-
[4] Beyer, 2024 Dataset (unfiltered)	49.14%	12.2155	0.3285	-
Average (Overall)	**53.78%**	**11.7314**	**0.4113**	**0.35**

Table 2 – Evaluations of both the best prototypical model (section 6.2.3) and the Obico Spaghetti Detective model (Obico, no date a) across all test subsets. Shows the model's accuracy, loss, F1 scores alongside image throughput on a Raspberry Pi 4B (4GB).

Across all test subsets my model achieves the best performance, with an average F1 score of 0.937 and an average image throughput on the raspberry pi of 15.10 images per second (Table 2). Comparing this to Obico's model which has more than a 40x lower throughput and achieving an average F1 score 2x lower than my network, it's evident that my approach significantly outperforms the de facto standard model.

The Obico's model performed best on datasets 2 and 3, which comprise mostly of Spaghetti related errors, however performed worse on datasets 1 and 4, large datasets with more samples where faults are less evident. This is most likely due to how the Spaghetti Detective model was only trained to detect spaghetti related issues (Jiang, 2019), however we find our model outperforms it even in these more challenging datasets, likely due to its broader training on diverse fault types, enabling it to generalise better across varying print scenarios.

7 Conclusion

7.1 Aim and Objectives

The original aim of this project was to "*produce a real-time, edge-deployable additive manufacturing fault detection system which can be inferred on common, low-cost devices*" (section 2.1). Through researching the most appropriate models and concluding that training a few-shot prototypical network with ShuffleNetv2 as the feature extractor is most effective on edge devices, I demonstrate that edge-based real-time AM fault detection systems are viable by producing a SOTA (state-of-the-art) open-source, edge deployable model. Additionally, my model used few-shot classification, completing my aim of reviewing if few-shot classification is viable for real-time edge deployment.

Objective One: Research existing additive manufacturing fault detection systems, identifying their strengths and limitations

My first objective was to investigate and build an understanding of what 3D print issues exist, evaluating their effect on the printed product and machines. In section 3.1, four of the most prevalent faults, stringing, improper bed adhesion, warping and layer splitting where analysed, finding improper bed adhesion and layer splitting to be the most prevalent issues which a small network (for edge devices) would have the best chance of learning.

Furthermore, I explored existing systems to evaluate their effectiveness, focusing on training techniques and how consumers use the models so that I can build on the successes of previous approaches while avoiding methods shown to be ineffective. I reviewed three existing fault detection systems (section 3.2) finding useful insights such that converting samples to greyscale allows for better generalisation as the colour of filament is not learnt, alongside the importance of data augmentation to introduce additional camera angles and lighting scenarios to aid generalisation.

Assessments into datasets were conducted (section 3.5) to assess whether they're mostly applicable for fine-tuning the CNN or training the network. Moreover, samples in the datasets were also evaluated to determine if they contained any artifacts that wouldn't be present in real-world scenarios such as graphics or text added in post which would need to be removed.

Objective Two: Investigate few-shot neural network architectures and techniques to allow real-time inference on edge devices

In section 3.3, the most prominent few-shot classification techniques were examined and compared, studying their performance on edge devices through their complexity alongside how accurately they perform. I found prototypical networks are much less computationally expensive to run (in comparison to matching networks) due to the fewer distance calculations needed to predict, as each class is represented by a single prototype, reducing the number of distance calculations required. I also concluded that prototypical networks should enable faster inference at a lower computational cost than classic CNNs for classification, as embedded distance calculations are less expensive than fully connected layers, furthering the initial hypothesis that few-shot classification networks can be used to produce real-time edge deployable defect detection systems.

Objective Three: Fine-tune a pre-trained image encoder for domain-specific feature extraction in 3D-printing image data

Research was conducted to evaluate computationally inexpensive CNNs (section 3.4), with a focus on their complexity and most importantly, performance on edge, ARM based devices. The findings concluded that ShuffleNetv2 was the most performant architecture to use due to its memory access cost optimisations, accelerating performance on ARM based devices without compromising on evaluation performance, despite its higher number of FLOPs.

Datasets were investigated ensuring there were a vast amount of camera angles, lighting scenarios and print types, to allow effective generalisation across the domain. Furthermore, transformations and augmentations were reviewed, finding augmentations such as flipping samples and converting them to greyscale can help improve the variety of datasets, aiding in generalisation. The effects of these transformations and augmentations were evaluated, finding them to increase training performance.

Research was conducted into techniques that could be used to combat model overfitting and underfitting using approaches such as dropout and layer freezing. Multiple different model configurations were trained and evaluated including freezing early layers as early layers tend to extract more general features, training all layers of the network alongside freezing the network and only training addition convolutions appended to it. Extensive training found that the best fine-tuned model was trained with initial layers frozen and small amounts of dropout used to prevent the model from overfitting.

Objective Four: Train and evaluate an additive manufacturing fault detection model utilising the previously established research and techniques

Leveraging the ShuffleNetv2 model which achieved the most performant results, multiple model configurations were evaluated when training the prototypical network. Models were trained and evaluated over different datasets to ensure the most applicable set was used during training. Investigations into the most capable network revealed fine-tuning the feature extractor exhibited worse results than without. Model performances were assessed across different unseen datasets, confirming that my model generalises well. The image throughput was tested on a Raspberry Pi 4B (4GB) which showed that it can run in real-time (15.1 images per second) on an edge device. Comparisons between it and the other available fault detection model (Obico Spaghetti Detective) demonstrated that my model outperforms it on all test datasets with over a 40x better image throughput and over 2x improved averaged F1-score.

7.2 Limitations and Future Work

Whilst my model generalises well across most faults, it's apparent from evaluations across varying datasets (Table 2) that it performs better on in-process prints in comparison to completed prints with defects (performing worse on dataset 2). This could be due to the training dataset containing less samples of completed prints however it's most likely due to how completed prints differ significantly in appearance whilst still printing samples contain similar properties, such as layering patterns and partial structural features indicative of the ongoing printing process. While the model is meant for real-time fault detection meaning it is less likely to be exposed to completed prints during operation, this is still a limiting factor of the model that could be improved.

My model performs binary classification, only labelling samples as defective or not. While the model was only trained to identify faults which could cause failures to machines and prints, it could be useful to recognise less destructive faults such as stringing and warping which would require the model to perform multi-class classification. As previously mentioned, prototypical networks perform adversely when there are more classes, however further work could be carried out to explore if multi-class classification could work for this model.

To utilise this model fully, it should be integrated into a real-time system which is capable of autonomously stopping prints when faults are identified, or notifying the user so further actions can be taken. Work into creating such a system should focus on embedding the model on edge devices through low-latency inference frameworks such as NCNN by Tencent or ONNX by Meta and Microsoft, which on most edge platforms increase model throughput. Additionally, to enhance reliability, a majority voting system across sequential frames could be implemented to reduce false positives and improve the consistency of detection results.

7.3 Summary

Overall, this project was successful in demonstrating the effectiveness of using a few-shot classification network with a lightweight feature extractor for classifying defective 3D-prints. Whilst fine-tuning strategies for the feature extractor were evaluated, utilising a pretrained model on ImageNet yielded better generalisation

when used to train the prototypical network. Datasets were analysed and curated ensuring varied camera angles and lighting scenarios to aid generalisation. Comparisons found significant improvements in image throughput (more than 40x faster on a Raspberry Pi 4B with 15.10 images per second) alongside precision and recall (2x average F1 score) against the other existing open model, achieving an F1 score of 0.937, making my model SOTA (state-of-the-art) for real-time open-source edge-deployable systems.

References

Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M. and Farhan, L., 2021. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. Journal of big Data, 8, pp.1-74.

Bambu Labs (no date) Bambu lab A1 Mini 3D printer, Bambu Lab Store. Available at: https://uk.store.bambulab.com/products/a1-mini (Accessed: 10 April 2025).

Beyer, N.H. (2024) 3D printing errors, Kaggle. Available at: https://www.kaggle.com/datasets/nimbus200/3d-printing-errors (Accessed: 10 April 2025).

Bhardwaj, S. (2025) 3D printing - success - failure dataset- FINETUNED, Kaggle. Available at: https://www.kaggle.com/datasets/bshaurya/3d-printing-success-failure-dataset-finetuned (Accessed: 07 March 2025).

ContextWorld (2024) Press releases, contextworld. Available at: https://www.contextworld.com/entry-level-3d-printer-shipments-skyrocket-globally-once-more-while-industrial-shipments-stagnate-again (Accessed: 05 March 2025).

Defects of 3d printing (2024) FDM object detection dataset and pre-trained model by defects of 3D printing, Roboflow. Available at: https://universe.roboflow.com/defects-of-3d-printing/fdm-sw9eb (Accessed: 11 March 2025).

Denton, E.L., Zaremba, W., Bruna, J., LeCun, Y. and Fergus, R., 2014. Exploiting linear structure within convolutional networks for efficient evaluation. Advances in neural information processing systems, 27.

HE, M. (2022) 3D printing errors, Kaggle. Available at: https://www.kaggle.com/datasets/mikulhe/3d-printing-errors (Accessed: 07 March 2025).

Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

Ibm (2024) What is few-shot learning?, IBM. Available at: https://www.ibm.com/think/topics/few-shot-learning (Accessed: 10 April 2025).

Jeong, H. and Yoo, J.H., 2022. 3D-EDM: Early detection model for 3D-printer faults. arXiv preprint arXiv:2203.12147.

Jiang, K. (2019) How to train a smart detective - aka behind-the-scenes glimpse at deep learning in 3D printing: Obico, Obico. Available at: https://www.obico.io/blog/2019/08/14/3d-printing-deep-learning-training/ (Accessed: 10 April 2025).

Kornblith, S., Shlens, J. and Le, Q.V., 2019. Do better imagenet models transfer better?. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 2661-2671).

Krogh, A. and Hertz, J., 1991. A simple weight decay can improve generalization. Advances in neural information processing systems, 4.

Lake, B.M., Salakhutdinov, R. and Tenenbaum, J.B. (2015) 'Human-level concept learning through probabilistic program induction', Science, 350(6266), pp. 1332–1338. doi:10.1126/science.aab3050.

Li, B. et al. (2018) '3D printing fault detection based on process data', Lecture Notes in Electrical Engineering, pp. 385–396. doi:10.1007/978-981-13-2291-4_38.

Li, D., Zhao, J. and Ma, J., 2022. Experimental studies on rock thin-section image classification by deep learning-based approaches. Mathematics, 10(13), p.2317.

Li, X., Chen, S., Hu, X. and Yang, J., 2019. Understanding the disharmony between dropout and batch normalization by variance shift. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 2682-2690).

Luo, L., Xiong, Y., Liu, Y. and Sun, X., 2019. Adaptive gradient methods with dynamic bound of learning rate. arXiv preprint arXiv:1902.09843.

Ma, N., Zhang, X., Zheng, H.T. and Sun, J., 2018. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In Proceedings of the European conference on computer vision (ECCV) (pp. 116-131).

MatterHackers (2025) 3D printer Troubleshooting Guide, MatterHackers. Available at: https://www.matterhackers.com/articles/3d-printer-troubleshooting-guide (Accessed: 10 April 2025).

Meftahmafazy (2023) Defect-3D-printing, Kaggle. Available at: https://www.kaggle.com/datasets/muhammadmeftahmafazy/defect-3d-printing (Accessed: 24 April 2025).

Niu, Y., Chadwick, E., Ma, A.W. and Yang, Q., 2023, September. Semi-Siamese Network for Robust Change Detection Across Different Domains with Applications to 3D Printing. In International Conference on Computer Vision Systems (pp. 183-196). Cham: Springer Nature Switzerland.

Obico (2025) The Obico Server, GitHub. Available at: https://github.com/TheSpaghettiDetective/obico-server (Accessed: 22 April 2025).

Obico (no date a) Obico Cloud Pricing & Plans - 3D printer monitoring, Obico. Available at: https://app.obico.io/ent_pub/pricing/ (Accessed: 10 April 2025).

Obico (no date b) Hardware requirements: Obico, Obico. Available at: https://www.obico.io/docs/server-guides/hardware-requirements/ (Accessed: 10 April 2025).

Projekt, A. (2024) 3D Dataset, Roboflow. Available at: https://universe.roboflow.com/automatisierung-projekt-aajut/3d-ho5at (Accessed: 12 April 2025).

Prusa Research (no date) Spaghetti monster, Prusa Knowledge Base. Available at: https://help.prusa3d.com/article/spaghetti-monster_1999 (Accessed: 10 April 2025).

Prusa Research (2024) Layer separation and splitting FDM, Prusa Knowledge Base. Available at: https://help.prusa3d.com/article/layer-separation-and-splitting-fdm_1806 (Accessed: 22 April 2025).

PyTorch Contributors (2024) Cross Entropy Loss, PyTorch 2.6 documentation. Available at: https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html (Accessed: 17 April 2025).

Pytorch Team (no date) Shufflenet v2, PyTorch. Available at: https://pytorch.org/hub/pytorch_vision_shufflenet_v2/ (Accessed: 16 April 2025).

QA, A.M. (2022) 3dprinting object detection dataset and pre-trained model by additive manufacturing QA, Roboflow. Available at: https://universe.roboflow.com/additive-manufacturing-qa/3dprinting (Accessed: 07 March 2025).

ROSeaboyer, IAdam1n and InternetArchiveBot (2024) List of macbook pros, The Apple Wiki. Available at: https://theapplewiki.com/wiki/List_of_MacBook_Pros (Accessed: 10 April 2025).

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C., 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510-4520).

SimplyPrint (2020) 3D printing: Stringing and how to fix it, SimplyPrint. Available at: https://simplyprint.io/print-troubleshoot/stringing (Accessed: 10 April 2025).

Simplify3D (2019a) Stringing or oozing, Simplify3D Software. Available at: https://www.simplify3d.com/resources/print-quality-troubleshooting/stringing-or-oozing/ (Accessed: 22 April 2025).

Simplify3D (2019b) Not sticking to the bed, Simplify3D Software. Available at: https://www.simplify3d.com/resources/print-quality-troubleshooting/not-sticking-to-the-bed/ (Accessed: 10 April 2025).

Simplify3D (2019c) Warping, Simplify3D Software. Available at: https://www.simplify3d.com/resources/print-quality-troubleshooting/warping/ (Accessed: 22 April 2025).

Snell, J., Swersky, K. and Zemel, R., 2017. Prototypical networks for few-shot learning. Advances in neural information processing systems, 30.

Snell, J. and Whitcomb, R. (2018) Prototypical Networks for Few-shot Learning, GitHub. Available at: https://github.com/jakesnell/prototypical-networks/ (Accessed: 17 April 2025).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1), pp.1929-1958.

Sun, H., Zhou, W., Yang, J., Shao, Y., Xing, L., Zhao, Q. and Zhang, L., 2024. An Improved Medical Image Classification Algorithm Based on Adam Optimizer. Mathematics, 12(16), p.2509.

Tian, Y., Wang, Y., Krishnan, D., Tenenbaum, J.B. and Isola, P., 2020. Rethinking few-shot image classification: a good embedding is all you need?. In Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16 (pp. 266-282). Springer International Publishing.

Vinyals, O., Blundell, C., Lillicrap, T. and Wierstra, D., 2016. Matching networks for one shot learning. Advances in neural information processing systems, 29.

QA, A.M. (2022) 3dprinting object detection dataset and pre-trained model by additive manufacturing QA, Roboflow. Available at: https://universe.roboflow.com/additive-manufacturing-qa/3dprinting (Accessed: 07 March 2025).

Xie, S., Girshick, R., Dollár, P., Tu, Z. and He, K., 2017. Aggregated residual transformations for deep neural networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1492-1500).

Yosinski, J., Clune, J., Bengio, Y. and Lipson, H., 2014. How transferable are features in deep neural networks?. Advances in neural information processing systems, 27.

Zhang, X., Zhou, X., Lin, M. and Sun, J., 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 6848-6856).

Appendices

Appendix A: Feature Extractor Evaluation Results

Table A1. Training results for Fig. 5 models, showing their best epochs training and validation sets loss and accuracies on the CNN dataset [2].

Model Name	Configuration	Best Epoch	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
[1] - *Standard*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = No	19	0.173	0.205	93.9%	92.8%
[2] – *Early Layers Frozen*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = [conv1, stage2]	30	0.120	0.177	95.2%	94.0%
[3] – *No Transformations*	Lr = 1e-4 Dropout = None Transforms = No Frozen Layers = No	8	0.241	0.176	96.0%	94.0%

Table A2. Training results for Fig. 6 models, showing their training and validation sets loss and accuracies for their best epochs on the CNN dataset [2].

Model Name	Configuration	Best Epoch	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
[1] – *High Dropout*	Lr = 1e-4 Dropout = [fc-0.5, conv5-0.5, stage4-0.05, stage3-0.025] Transforms = Yes Frozen Layers = No	100	0.299	0.140	81.5%	93.6%
[2] – *Low Dropout*	Lr = 1e-4 Dropout = [fc-0.4, conv5-0.3, stage4-0.05, stage3-0.025] Transforms = Yes Frozen Layers = No	100	0.270	0.149	85.0%	95.2%

Table A3. Training results for Fig. 7 models, showing their training and validation sets loss and accuracies for their best epochs on the CNN dataset [2].

Model Name	Configuration	Best Epoch	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
[1] – *Low Dropout*	Lr = 1e-4 Dropout = [fc-0.2, conv5-0.15, stage4-0.01, stage3-0.01] Transforms = Yes	65	0.142	0.188	93.2%	94.0%

	Frozen Layers = No					
[2] – *Lowest Dropout*	Lr = 1e-4 Dropout = [fc-0.1, conv5-0.1, stage4-0.01, stage3-0.01] Transforms = Yes Frozen Layers = [conv1, stage2]	63	0.155	0.162	93.1%	94.0%

Table A4. Training results for Fig. 8 models, showing their training and validation sets loss and accuracies on the CNN dataset [2] for their best epochs, where additional layers are described with their output channels (i.e. conv (1024) represents a convolution with 1024 output channels), bn is batch normalisation and ReLU is activation.

Model Name	**Configuration**	**Best Epoch**	**Training Loss**	**Validation Loss**	**Training Accuracy**	**Validation Accuracy**
[1] – *1024*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = [conv1,stages 2-4, conv5] Additional Layers: [conv (1024), bn, ReLU]	24	0.210	0.242	91.5%	92.0%
[2] – *128*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = [conv1,stages 2-4, conv5] Additional Layers: [conv (128), bn, ReLU]	24	0.282	0.279	88.5%	90.0%

Table A5. Training results for Fig. 9 models, showing their training and validation sets loss and accuracies for their best epochs on the CNN dataset [2], where additional layers are described with their output channels (i.e. conv (1024) represents a convolution with 1024 output channels), bn is batch normalisation and ReLU is activation.

Model Name	**Configuration**	**Best Epoch**	**Training Loss**	**Validation Loss**	**Training Accuracy**	**Validation Accuracy**
[1] – *64x64*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = [conv1,stages 2-4, conv5] Additional Layers: [conv (64), bn, ReLU, conv (64), bn, ReLU]	100	0.171	0.218	93.2%	93.2%
[2] – *64x32*	Lr = 1e-4 Dropout = None Transforms = Yes Frozen Layers = [conv1,stages 2-4, conv5]	100	0.180	0.234	93.1%	91.6%

	Additional Layers: [conv (64), bn, ReLU, conv (32), bn, ReLU]					

Table A6. Training results for Fig. 10, showing models training and validation sets loss and accuracies for their best epochs on the CNN dataset [2], where additional layers are described with their output channels (i.e. conv (1024) represents a convolution with 1024 output channels), bn is batch normalisation and ReLU is activation.

Model Name	**Configuration**	**Best Epoch**	**Training Loss**	**Validation Loss**	**Training Accuracy**	**Validation Accuracy**
[1] – *64x64-dropout*	Lr = 1e-4 Dropout = [0.4 after all additional convolutions] Transforms = Yes Frozen Layers = [conv1,stages 2-4, conv5] Additional Layers: [conv (64), bn, ReLU, conv (64), bn, ReLU]	100	0.220	0.244	91.6%	92.0%

Appendix B: Prototypical Network Evaluation Results

Table B1. Evaluation results for Fig. 11. Shows the loss and accuracy of a model over all datasets.

		Datasets							
		[1] Prototypical Network		[2] CNN Dataset		[3] Prototypical Network (reduced)		[4] Beyer (2024) Dataset (unfiltered)	
Model Name	**Configuration**	**Loss**	**Acc.**	**Loss**	**Acc.**	**Loss**	**Acc.**	**Loss**	**Acc.**
[1] *Fine-Tuned CNN*	Lr = 1e-5 Epoch = 50 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0 Dataset = [1]	0.035	99.2	0.317	91.2	0.216	90.7	0.400	83.5
[2] *Fine-Tuned CNN*	Lr = 1e-5 Epoch = 50 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0 Dataset = [2]	0.666	67.9	0.006	86.7	0.976	63.5	0.711	54.1
[3] *Fine-Tuned CNN (reduced)*	Lr = 1e-6 Epoch = 50 Dropout = None Shots = 3 Test Query = 5 Episodes = 100	0.859	71.5	0.718	86.7	0.021	99.8	0.807	59.0

	Weight Decay = 0.0 Dataset = [3]								

Table B2. Evaluation results for Fig. 12 showing the training and validation loss and accuracies for the models on the prototypical network dataset [1].

Model Name	**Configuration**	**Best Epoch**	**Training Loss**	**Validation Loss**	**Training Accuracy**	**Validation Accuracy**
[1] – *lr = 1e-4*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0	10	0.003	0.001	100%	100%
[2] – *lr = 1e-4,* *wd = 1e-5*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 1e-5	7	0.012	0.008	99.9%	99.9%
[3] – *lr = 1e-5*	Lr = 1e-5 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0	25	0.058	0.034	98.2%	99.3%

Table B3. Evaluation results for Fig. 13. Shows the loss and accuracies over all datasets for each model.

		Datasets							
		[1] Prototypical Network		[2] CNN Dataset		[3] Prototypical Network (reduced)		[4] Beyer (2024) Dataset (unfiltered)	
Model Name	**Configuration**	**Loss**	**Acc.**	**Loss**	**Acc.**	**Loss**	**Acc.**	**Loss**	**Acc.**
[1] *Standard CNN*	Lr = 1e-5 Epoch = 30 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0 Dataset = [1]	0.040	99.5	0.376	85.5	0.106	97.1	0.226	92.4
[2] *Standard CNN*	Lr = 1e-5 Epoch = 50 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0 Dataset = [2]	0.667	60.7	0.016	100	0.714	65.4	0.690	56.2

[3] *Standard CNN*	Lr = 1e-5 Epoch = 50 Dropout = None Shots = 3 Test Query = 5 Episodes = 100 Weight Decay = 0.0 Dataset = [3]	0.667	61.1	0.660	69.5	0.021	100	0.687	55.3

Table B4. Evaluation results for Fig. 14. Shows the training and validation loss and accuracies for the models on the prototypical network dataset [1].

Model Name	**Configuration**	**Best Epoch**	**Training Loss**	**Validation Loss**	**Training Accuracy**	**Validation Accuracy**
[1] – *lr = 1e-4*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0	10	0.016	0.006	99.7%	99.9%
[2] – *lr = 1e-5*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 1e-5	30	0.047	0.036	99.5%	99.5%
[3] – *lr = 1e-5 (dropout)*	Lr = 1e-5 Dropout = [fc, conv5, stage3, stage4 all 0.01] Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 0.0	50	0.152	0.070	95.0%	98.1%

Fig. B5. Validation and training set evaluations for prototypical network models trained using the non-fine-tuned feature extractor with weight decay. Plotting the model's accuracy (left) and loss (right).

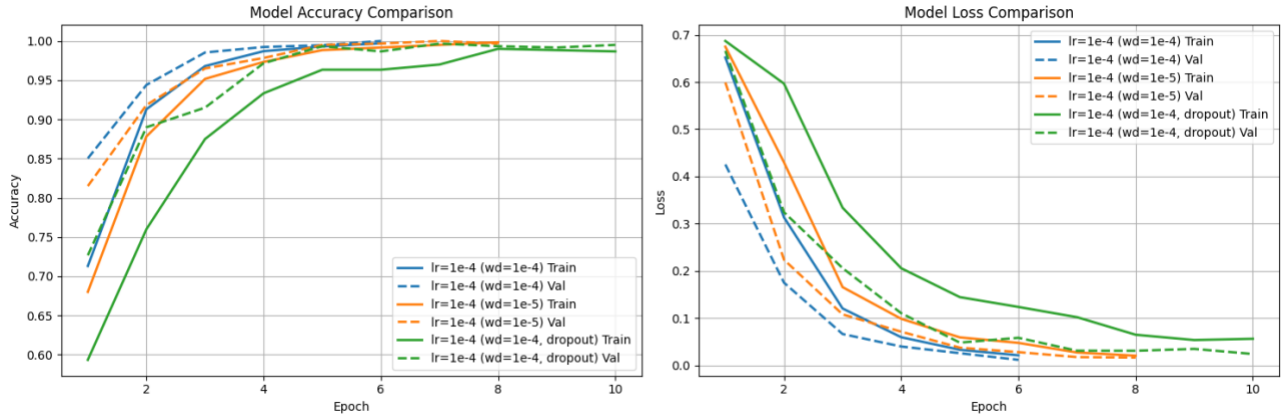

Table B6. Evaluations results for Fig. B5. Shows the training and validation loss and accuracies on the prototypical network dataset [1].

Model Name	Configuration	Best Epoch	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
[1] – *lr = 1e-4 (wd=1e-4)*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 1e-4	6	0.021	0.011	99.7%	100%
[2] – *lr = 1e-4 (wd=1e-4, dropout)*	Lr = 1e-4 Dropout = [fc, conv5, stage4 and stage3 all 0.01] Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 1e-4	10	0.056	0.024	98.7%	99.5%
[3] – *lr = 1e-4 (wd=1e-5)*	Lr = 1e-4 Dropout = None Shots = 5 Test Query = 15 Episodes = 100 Weight Decay = 1e-5	8	0.020	0.016	99.8%	99.7%